

Conceitos e Técnicas HTML

WBA0182_V1.1

Conceitos e Técnicas HTML

Autor: Rogério Carlos dos Santos

Como citar este documento: SANTOS, Rogério Carlos. **Conceitos e Técnicas HTML**. Valinhos: 2016.

Sumário

Apresentação da Disciplina	04
Unidade 1: Introdução à *web standards*	06
Assista a suas aulas	19
Unidade 2: Indo além da linguagem de marcação	27
Assista a suas aulas	41
Unidade 3: Estrutura básica de um documento HTML e como podemos trabalhar com XHTML	48
Assista a suas aulas	64
Unidade 4: Trabalhando com CSS	72
Assista a suas aulas	87

Conceitos e Técnicas HTML

Autor: Rogério Carlos dos Santos

Como citar este documento: SANTOS, Rogério Carlos. **Conceitos e Técnicas HTML**. Valinhos: 2016.

Sumário

Unidade 5: Desvendando o universo Javascript	95
Assista a suas aulas	110
Unidade 6: Trabalhando com HTML5	118
Assista a suas aulas	131
Unidade 7: Recursos avançados de HTML5	139
Assista a suas aulas	160
Unidade 8: Trabalhando com metadata e semântica	168
Assista a suas aulas	180

Apresentação da Disciplina

A internet foi planejada desde o seu início como uma tecnologia que interliga computadores para troca de conteúdos dos mais variados tipos. Para tornar isto possível, houve a necessidade da padronização de linguagens e regras de construção para que os *sites* pudessem ser lidos pelos diversos tipos de computadores existentes.

Nesta disciplina, você irá conhecer estas padronizações e as principais linguagens que tornam o uso da internet possível, tanto para computadores de grande porte, microcomputadores e os atuais **mobiles web, smartphones e tablets,** abrindo o caminho para que você se torne um profissional bem-sucedido provido de conhecimento sobre o assunto.

Você irá conhecer *Web Standards*, que são os conceitos desta padronização da internet, e que são adotados mundialmente pelos desenvolvedores de tecnologia voltada para a *Web*. Linguagens como HTML, SGML, XML, XHTML, suas atuais versões e como trabalhar com a construção de *sites* através de recursos, tais como CSS e responsividade, são informações que você estudará e que servirão de base para entender como os *sites* são construídos. Finalizando, você verá quais recursos são oferecidos pelo HTML5 na construção de *sites*, metadatas, semântica, formulários, áudio, vídeo e muitos outros que proporcionam a integração com o que há de mais atual na *Web*.

Através do estudo destes temas, você terá os principais conhecimentos para poder

trabalhar corretamente com a construção de *sites* e sua integração com as demais tecnologias *Web* existentes.

Unidade 1

Introdução à *web standards*

Objetivos

Este módulo irá mostrar o que é *web standards*, como e por quem são recomendadas. Serão apresentados os seguintes tópicos:

1. O que são *web standards*.
2. Internacionalização da *web standards*.
3. Motores de renderização.
4. Conceitos de interoperabilidade e usabilidade.

Introdução

A internet, também conhecida por Rede Mundial de Computadores, numa tradução livre do termo inglês "*World Wide Web*", representa o local onde todos podem compartilhar informações dos mais variados tipos, textos, imagens, vídeos, músicas, informações pessoais/corporativas, mensagens eletrônicas, comunicações por texto, som e por vídeo, documentos diversos, sentimentos e ideias através das inúmeras redes sociais existentes, comércio eletrônico e muito mais.

Para acessar estes conteúdos, diversos tipos de equipamentos podem ser utilizados, desde que façam uso de uma conexão de rede e um navegador ("***Browser***", em inglês).

Para que todas as tecnologias possam compartilhar este conteúdo mantendo um padrão de apresentação e uso, através dos diversos tipos computadores existentes, de seus sistemas operacionais e dos navegadores utilizados, houve, desde o início da internet, um esforço mundial para a padronização das tecnologias envolvidas no processo de desenvolvimento, tanto dos equipamentos, protocolos de rede e também das linguagens e sistemas adotados. Por todos estes motivos surgiram as *web standards*.

1. O que são *web standards*

1.1. Conceitos básicos

O W3C, ***World Wide Web Consortium,*** é responsável pelas recomendações de padrões de uso para a *Web*. Estas recomendações são chamadas de *Web Standards*, termo inglês que em uma tradução livre significa "padrões *Web*". O **W3C** é um consórcio internacional e conta com a participação de organizações públicas e privadas de desenvolvimento de tecnologia e é a responsável por mais de uma centena de padrões usados na *web,* como o HTML, CSS, SVG e HTML5. Estes padrões são abertos, gratuitos e têm a finalidade de manter a padronização para que a própria *web* possa funcionar e evoluir, mantendo sua interoperabilidade e usabilidade (3WC, s.d.).

Para saber mais

Desde seu início, a internet é acessada por navegadores desenvolvidos por empresas de tecnologia. Para que possa haver uma uniformidade no funcionamento destes navegadores e como eles apresentam os sites e as diversas tecnologias usadas para suas construções, o W3C desenvolve especificações técnicas e orientações com esta finalidade.

1.2. Padrões recomendados

O W3C mantém atualizada uma lista dos **padrões recomendados** para o desenvolvimento de tecnologia para a *web*. Estes padrões englobam recomendações para *Web*

Design, aplicações *web*, arquitetura *web*, semântica, XML, *web* de serviços e de dispositivos, recomendações para criação de novos navegadores e muitas outras. É importante que você conheça estes padrões, pois o sucesso de futuros projetos com certeza passa por estas recomendações (3WC, s.d.)

Link

Para que você conheça as principais recomendações, bem como visualize a lista completa delas, acesse: <http://www.w3c.br/Padroes/>. Acesso em: 11 set. 2016.

2. Internacionalização das *web standards*

A internacionalização de *web standards* diz respeito a caracteres especiais usados pelos diversos idiomas mundiais. Os desenvolvedores de *sites* e *web designers* devem sempre ter em mente as diferenças entre os idiomas e o uso de caracteres especiais que devem ser apresentados de forma correta pelos diversos navegadores. Assim, há diversas recomendações apresentadas pelo 3WC no sentido de padronizar o uso destes caracteres através de linguagens como o HTML, XHTML e outras. Estes caracteres especiais, também chamados de entidades, são aceitos pelo HTML e suas linguagens correlatas como XHTML e HTML5 e formam conjuntos

de símbolos especiais. Existem três conjuntos principais:

- Caracteres ISO8859-1: responsáveis pelos caracteres latinos.
- Caracteres matemáticos, símbolos, letras latinas e gregas.
- Caracteres de marcação.

Para que estes caracteres possam ser mostrados de forma correta, cada um deles recebe um código numérico e é inserido na própria linguagem HTML para que possa ser interpretado de forma correta pelos navegadores (HOLZSCHLAG, 2004).

Para saber mais

O W3C desenvolve especificações técnicas e orientações para o bom funcionamento e a padronização da *web*. As regras de internacionalização estão descritas pelo 3WC. No caso de caracteres especiais, por exemplo a fração matemática de "⅗" (três quintos) será escrita no código HTML como "⅗" (3WC).

Link

O W3C disponibiliza a descrição de regras de internacionalização. Disponível em: <https://www.w3.org/TR/its20//>. Acesso em: 11 set. 2016.

3. Motores de renderização

Os navegadores de internet, chamados de *Browser*, contêm mecanismos que interpretam o código HTML e os exibem. Estes mecanismos são chamados de motores de renderização. É possível ter uma noção do quanto seu *site* será apresentado corretamente, de acordo com o motor escolhido e sua abrangência de mercado. Por exemplo, usando-se um código compatível com o motor de renderização do Google Chrome, tem-se uma ideia de quantas pessoas serão impactadas, dependendo do local, público ou da plataforma tecnológica (FLATSCHART, 2011).

Temos no quadro a seguir os principais **motores e seus respectivos *browsers***:

Quadro 1 | Motores e seus *browsers*

Motor	Desenvolvedor	Principais Aplicações
Blink	Google, Opera, Samsung, Intel e outros	Google Chrome e Opera
Gecko	Nestscape e Mozila	Mozila Firefox
KHTML	KDE	Konqueror
Presto	Opera Software	Opera (até v. 14)
Trident	Microsoft	Internet Explorer
Webkit	Apple, KDE, Nokia, Palm e outros	Apple Safari

Fonte: Flatschart (2011).

Link

Saiba quais comandos são compatíveis em cada *browser* e seus respectivos motores de renderização. Disponível em: <http://caniuse.com/>. Acesso em: 11 set. 2016

4. Conceitos de interoperabilidade e usabilidade

4.1 Interoperabilidade

Quando falamos de tecnologia de internet, como a *web* funciona e todos os sistemas e linguagens usados, além das recomendações de desenvolvimento, as chamadas *web standards*, existe o conceito de interoperabilidade.

A interoperabilidade na prática é a possibilidade de diferentes sistemas se comunicarem e trocarem informações e dados entre si. Para que isto aconteça, é necessário que se observe ao máximo todas as recomendações de padronização propostas tanto em protocolos de redes de computadores adotados, como pelos sistemas desenvolvidos para funcionarem na internet, mantendo a compatibilidade com os diversos navegadores e as regras e padrões propostos pelo 3WC referente às linguagens adotadas como HTML, XHTML e outras linguagens e padrões. Estes padrões devem ser abertos para que todas as tecnologias e linguagens possam utilizá-los e assim contribuir para uma interoperabilidade cada vez maior entre os diversos sistemas (ALBINADER NETO;

LINS, 2006).

4.2. Usabilidade

O termo **usabilidade** é usado para definir a facilidade de uso que as pessoas podem ter quando usarem um determinado *site* ou programa. Normalmente pode-se dizer que um *site* tem uma boa usabilidade quando estando dentro de uma página deste *site* temos a possibilidade de ir a qualquer outra parte deste *site* com apenas um clique ou toque, sem ter que usar o botão voltar. Existem métodos para testar esta eficiência de usabilidade, como, por exemplo, a Análise Heurística, que é um método que analisa um determinado site de acordo com regras pré-estabelecidas, verificando se o usuário encontrará uma maior ou menor facilidade de uso para um *site* ou sistema. Na prática, a Análise Heurística submete a interface de um determinado aplicativo a avaliação de especialistas em **usabilidade** (NIELSEN; HOA, 2007).

Para saber mais

A usabilidade é um conceito extremamente importante no desenvolvimento de tecnologias, tanto na *web* como em sistemas. As empresas mantêm uma pesquisa contínua de usabilidade visando sempre melhorar seus produtos e, consequentemente, facilitar o uso de sua tecnologia para todas as pessoas.

Link

Verifique o artigo sobre usabilidade, *Experiência do usuário, usabilidade e design*, disponibilizado pela empresa IBM. Disponível em: <https://www.ibm.com/developerworks/community/blogs/tlcbr/entry/mp260?lang=en->. Acesso em: 11 set. 2016.

Glossário

Browser: *software*/programa que permite ao usuário navegar pela internet abrindo as páginas e sites de seu interesse, visualizando seus conteúdos (MEUSDICIONÁRIOS, 2016).

Mobiles web, smartphones e tablets: aparelhos de tecnologia móvel, podem ser carregados pelas pessoas a qualquer lugar e acessam a internet por conexões *wi-fi* ou por chamadas de telefonia celular 3G ou 4G (GARDNER; GRIGSBY, 2013).

World Wide Web Consortium: também chamado de W3C, desenvolve especificações técnicas e orientações através de um processo projetado para maximizar a consenso sobre as recomendações, garantindo qualidades técnicas e editoriais, além de transparentemente alcançar apoio da comunidade de desenvolvedores, do consórcio e do público em geral (3WC, s.d.).

Questão para reflexão



Você viu neste módulo que existem padrões adotados e que esta padronização é atualmente administrada pelo 3WC, que é o consórcio internacional de empresas e instituições que contribuem e se utilizam destes padrões. Descreva através de uma pesquisa, quais linguagens usadas na internet adotam estes padrões, como estas especificações técnicas são classificadas, e, em especial, pesquise sobre a padronização dos *WEB SERVICES,* pois isto o ajudará a entender melhor as *web standards*.

Considerações Finais

Neste módulo, você aprendeu:

- O que são *web standards*.
- Conceitos básicos de *web standards*.
- Padrões recomendados para *web standards*.
- Internacionalização das *web standards*.
- Motores de renderização.
- Conceitos de interoperabilidade e usabilidade.

Referências

3WC – site 3WC. Disponível em: <http://www.w3c.br/Padroes/>. Acesso em: 11 set. 2016.

ALBINADER NETO, Jorge Abílio; LINS, Dueire. **Web services em Java**. Rio de Janeiro: Brasport. 2006.

EIS, Diego; FERREIRA, Elcio. **HTML5 e CSS3 com farinha e pimenta**. São Paulo: Tableless, 2012.

FLATSCHART, Fábio. **HTML5**: embarque imediato. Rio de Janeiro: Brasport, 2011.

GARDNER, Lysa; GRIGSBY, Jason. **Use a cabeça**! Mobile Web – Starlin Alta Editora e Consultoria Eireli. 2013.

HOLZSCHLAG, Moly E. 250 **segredos para web designers**. Tradução de Marcos Vieira – Rio de Janeiro: Elsevier. 2004.

MEUSDICIONARIOS. Disponível em: <http://www.meusdicionarios.com.br/browser>. Acesso em: 11 set. 2016.

NIELSEN, Jacob; HOA, Loranger. **Usabilidade na web**. Tradução Edson Fumankiewicz & Carlos Schafranski. Rio de Janeiro: Elsevier, 2007.

Assista a suas aulas



Aula 1 - Tema: Introdução ao *web standards*. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/41c155acee3866f993057bf102b275e7>.



Aula 1 - Tema: Introdução ao *web standards*. Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/7cda77436630fd0dc1eee6b53635732c>.

Questão 1

1. Assinale a alternativa que contempla o que são as *web standards*.

a) Padrões para desenvolvimento de *Web Design* apenas.

b) Padrões e recomendações exclusivos de comunicação em rede usados na *web*.

c) Descrições de linguagens aceitas pelos navegadores de internet.

d) Padrões usados na web, como o HTML, CSS, SVG, HTML5 e muitos outros.

e) Padrões de funcionamento dos navegadores de internet que iguala navegadores de computadores e navegadores de aparelhos *smartphones* e *tablets*.

Questão 2

2. Assinale a alternativa que contempla os motores de renderização utilizados:

a) Blink, Gecko, KHTML, Presto, Trident e Webikit.

b) IOS, Gecko, KHTML, XHTML, Webkit e Android.

c) Android, Gecko, KHTML, Konqueror e Webikit.

d) Gecko, Opera, KHTML, Trident, Blink, e Webikit.

e) Presto, Blink, Gecko, XML, Webikit e Trident.

Questão 3

3. Assinale a alternativa que explica a interoperabilidade entre sistemas.

a) A capacidade de um sistema de ser operado local ou remotamente.

b) A capacidade de um sistema poder ser usado por qualquer sistema operacional ou computador.

c) A possibilidade de diferentes sistemas se comunicarem e trocar informações e dados entre si.

d) A capacidade dos sistemas de acessarem a *web*.

e) A padronização internacional de linguagens e protocolos web.

Questão 4

4. Os padrões de internacionalização das *web standards* preveem conjuntos de caracteres especiais usados pelos diversos idiomas existentes. São conjuntos de caracteres previstos pelas *web standards*:

a) Caracteres ISO8859-1, ISO9001-8859 e caracteres matemáticos.

b) Caracteres ISO8859-1, caracteres matemáticos e caracteres de marcação.

c) Caracteres ISO8859-1, caracteres gráficos e caracteres ISO7070.

d) Caracteres ISO8859-1, caracteres matemáticos e caracteres ASCII.

e) Caracteres ISO8859-1, caracteres UNICODE e caracteres de ASCII.

Questão 5

5. O conceito de usabilidade define:

a) Como um programa será usado em determinado sistema operacional.

b) Como as linguagens de web são usadas pelos navegadores.

c) Como os navegadores de internet são usados pelos diferentes sistemas operacionais.

d) O estudo de novas funções para que os usuários possam usar.

e) A facilidade de uso que as pessoas podem ter quando forem usar um determinado site ou programa.

Gabarito

1. Resposta: D.

As *web standards* são recomendações e padrões adotados pelas linguagens usadas no desenvolvimento *web*, por sistemas que se interligam na *web*, padronização dos protocolos de envio e recebimento de dados e muitos outros. Assim, é incorreto dizer que estes padrões atendem unicamente a uma tecnologia ou linguagem, e em função disto as demais alternativas estão incorretas.

2. Resposta: A.

Nas demais opções temos itens que não são motores de renderização, como por exemplo Android e IOS que são sistemas operacionais ou XML ou XHTML que são linguagens de descrição de página.

3. Resposta: C.

As outras opções descrevem capacidades ou funcionalidades que, ou não existem, ou que não descrevem o que é interoperabilidade.

4. Resposta: B.

Nas demais questões são citados padrões inexistentes, ou que não são padrões usados na internacionalização, como os padrões UNICODE e ASCII, por exemplo.

5. Resposta: E.

Nas outras opções encontramos definições incorretas para usabilidade, como por exemplo, usabilidade. Estudar como um programa será usado por um sistema operacional e estudo de novas funções de um programa.

Unidade 2

Indo além da linguagem de marcação

Objetivos

Neste módulo, você conhecerá as linguagens utilizadas para desenvolver *sites* na *web* e seus princípios básicos. São divididos em:

1. Linguagem SGML.
2. Linguagem HTML.
3. Linguagem XML.
4. Versões de HTML E XHTML.

Introdução

O acesso de conteúdos compartilhados na rede mundial de computadores, também conhecida por *World Wide Web*, ou simplesmente "www", é realizado pelo uso de navegadores (*Browser,* em inglês) que na prática se utilizam de **hipertexto** para poder elencar as diversas páginas e *sites* através de um simples clique do *mouse* por parte dos seus usuários. Também conhecido como *hiperlink,* ou *links* como são popularmente chamados, que proporcionaram a facilidade de uso da internet e serviram de base para as linguagens utilizadas pelos navegadores (COSTA, 2007).

Tim Berners Lee, quem em 1990 trabalhava no Centro Europeu de Pesquisa Nuclear – CERN – na Suíça, inventou o Protocolo HTTP e a linguagem HTML, ambos com objetivo de permitir a troca de conteúdos entre diversos centros de pesquisa utilizando a internet. Para isso, cada texto teria uma etiqueta (tag) que representaria um *link* entre dois documentos ou dois pontos de um mesmo documento, estabelecendo assim a ideia de hipertexto (FRONTEND, 2016).

Pode-se dizer que o HTML foi baseado na estrutura da linguagem SGML.

Temos ainda a evolução cronológica dessas linguagens conforme a sequência:

SGML → HTML 1.0 → HTML xx → XML → HTML 4.01 → XHTML.

Link

Conheça mais sobre hipertexto em: <http://www.webartigos.com/artigos/o-que-e-hipertexto/63870>. Acesso em: 11 set. 2016.

1. Linguagem SGML

SGML é uma sigla do inglês "*Standard Generalized Markup Language*". A linguagem SGML foi desenvolvida em 1960 pela IBM para permitir a classificação e o compartilhamento de grande quantidade de documentos texto, e utilizava-se de etiquetas (***tags***) que pudessem relacionar de forma estruturada estes documentos.

Atualmente, a linguagem **SGML** tornou-se uma norma ISO 8879. A SGML tem um conjunto de *tags*, também denominados por elementos, e permite que sejam criadas novas *tags* conforme desejado. Isto serviu de base para as linguagens de marcação como o HTML e posteriormente o XML (IBM K, s.d.).

Para saber mais

Um exemplo de uma SGML

```
<QUOTE TYPE="EXEMPLO DE SGML"
    Típico exemplo de utilização de
uma tag <BOLD> este texto será ne-
                        grito </BOLD>
                                </QUOTE>
```

Observe que por definição, uma tag quando aberta deve ser ao final fechada, "<tag> </tag>", sendo uma das principais características de uma linguagem de marcação.

2. Linguagem HTML

O **HTML**, sigla do inglês "*Hypertext Markup Language*", é uma linguagem de marcação de hipertexto voltada para a estruturação de documentos que serão apresentados em um navegador de internet *("browser").* Na prática, isto significa que a linguagem HTML é utilizada para construir *sites* para a internet. Ela serve também como base para integrar outras tecnologias utilizadas na *web*.

A linguagem HTML foi criada por Tim Berners Lee, no ano de 1990, foi baseada na linguagem SGML, para a composição e apresentação de documentos na *web*. Para isto, o HTML utiliza elementos (*tags*) agrupados por funções e mantém uma estrutura básica para a utilização destes elementos.

Inicialmente, os padrões adotados para os elementos em HTML foram amplamente discutidos e em 1993 o organismo internacional "IETF" (*Internet Engineering Task Force*) apresentou a primeira proposta de padronização da linguagem. Foram definidas as regras para as *tags* de imagens, tabelas, formulários e outros. Em 1995, o IETF, através de seu grupo de trabalho, organizou as regras para o padrão HTML 2.0, que apesar do nome foi o primeiro padrão oficial desta linguagem. Já em 1996 foi estabelecido o "*3WC – World Wide Web Consortium",* que orienta a padronização desta linguagem até os dias atuais. (W3 HTML)

Para saber mais

Observe o exemplo de trecho de código em HTML. Os elementos/tags são descritos entre "< >" e "</ >" e interpretados pelo navegador web (*browser*)

```
<html lang="pt-br">
<head>
    <meta charset="utf-8">
    <title>Título da página</title>
</head>
```

Link

Uma descrição dos elementos/tags utilizados atualmente pela linguagem HTML pode ser encontrada em: <https://developer.mozilla.org/pt-BR/docs/Web/HTML/HTML5/HTML5_element_list/>. Acesso em: 11 set. 2016.

3. Linguagem XML

A linguagem XML, Sigla do inglês "*Extensible Markup Language*", é uma linguagem de marcação baseada na linguagem SGML. Além do objetivo de proporcionar mais flexibilidade nos elementos/tags utilizados para a construção de conteúdo para *web*, o XML conseguiu estabelecer uma melhor padro-

nização entre os diversos navegadores dos diversos sistemas operacionais utilizados para acessar a *web*. Além disso, esta linguagem, por permitir a definição de novos elementos/tags, facilita a criação de novas estruturas de documentos, proporciona uma maior legibilidade e entendimento por pessoas menos acostumadas com linguagens de marcação e, ainda, facilita a análise de documentos XML por programas.

Mas nem tudo é tão perfeito assim na linguagem XML original. Por não possuir um conjunto pré-definido de *tags* ou elementos, o XML não provê mecanismos para apresentação de dados nem para ligação entre documentos, ou seja, não tinha recursos de *links* entre as páginas, por exemplo. É necessário utilizar o **XML** juntamente com outras linguagens, como o HTML, para resolver estes problemas. Desde 1998, o XML tornou-se um padrão e que segue as recomendações do 3WC.

Para saber mais

Observe o exemplo de código em XML. Mesmo você não tendo maiores conhecimentos na linguagem XML, este código apresenta-se plenamente legível.

```
<addressBook>
  <person gender="male">
    <name>
      <family>Wallace</family>
<given>Bob</given>
    </name>
    <email>bwallace@megacorp.com</
email>
  </person>
```

Fonte: IBM, 2016.

Link

Conheça mais sobre a linguagem XML através deste tutorial. Disponível em: <http://www.ibm.com/developerworks/xml/newto/>. Acesso: 11 set. 2016.

4. HTML e XHTML

4.1 HTML

Como já foi visto, o HTML, ou "*Hypertext Markup Language*", é uma Linguagem de marcação de hipertexto voltada para a estruturação de documentos que serão apresentados em um navegador de internet, é utilizada para construir *sites* para a internet e serve como base para integrar outras tecnologias utilizadas na *web*.

Dessa forma, desde 1996, o 3WC orienta as especificações da linguagem HTML, cuidando de sua padronização e evolução para que se possa assimilar as evoluções tecnológicas da própria *web*. Dessa forma, houve várias atualizações do HTML, como o HTML 2.0, HTML 3.2, HTML 4.0, e ainda uma subversão, o HTML 4.01. As recomendações do HTML 4.0 foram publicadas pelo W3C, em 18 de dezembro de 1997 e revistas no HTML 4.01, em 24 de abril de 1998, em conformidade com a norma ISO 8879.

O W3C ainda recomendou que os navegadores de internet e demais ferramentas de navegação continuassem a suportar as versões anteriores HTML 3.2 e HTML 2.0.

A linguagem HTML é utilizada até hoje e é considerada como a base de todas as demais tecnologias empregadas na construção de *sites*. Ela também serviu de base para a criação da linguagem XHTML e recentemente para a linguagem HTML 5 (W3 HTML, s.d.).

4.2 DTD – Document Type Definition

A estruturação das linguagens de marcação prevê a utilização de uma **DTD**, sigla do inglês "*Document Type Definition*", ou Definição do Tipo do Documento, particularmente chamados também de "*Doctypes*" ou tipo de documento. Estes DTDs definem na prática qual tipo de documento está sendo interpretado pelo navegador, tornando possível para estes programas entenderem qual

é o tipo de tecnologia usada na construção de determinado *site*.



Link

As regras para a aplicação de DTD (em inglês, pode ser traduzido pelo *plug-in* do Google Chrome) podem ser vistas em: <https://www.w3.org/TR/xhtml1/dtds.html>. Acesso em: 11 set. 2016.

4.3 XHTML

A linguagem XHTML, do inglês "*Extensible HyperText Markup Language*", foi a recomendação posterior ao HTML 4.01 e suas versões anteriores dadas pelo W3C. Na prática, é a junção do HTML e sua estrutura com a flexibilidade e as diversas funcionalidades do XML, ou seja, ele combina as *tags* do HTML com as regras do XML. Por estes motivos o XHTML é considerado o sucessor do HTML. O XHTML 1.0 foi a primeira versão desta nova linguagem e a primeira grande alteração desde o HTML 4.01. Formalmente, o XHTML é uma linguagem que possui elementos/tags que ampliam o HTML e o reformula em XML (PORTALEDUCAÇÃO, 2016).

O objetivo é que o XHTML possa ser interpretado por qualquer navegador em qualquer dispositivo, sejam microcomputadores, *smartphones* ou *tablets*, ou outros que possam através de seus dispositivos acessar a *web*.

Para que o XHTML possa suportar as tecnologias anteriores a ele em HTML, estabele-

ceu-se o uso de 3 tipos de "***Doctype***":

- ***Strict:*** não admite o uso de itens de formatação em seus elementos e também elementos em "desuso" segundo as recomendações do W3C. As formatações são feitas obrigatoriamente através do uso de Páginas de Estilo – CSS.
- ***Transitional:*** opção mais flexível para o uso do XHTML, permite a utilização de elementos em "desuso" e é indicada para aproveitar códigos originalmente escritos em HTML, regras de formatação embutidas em *tags* e a apresentação de documentos sem o uso de CSS.
- ***Frameset:*** igual ao *Transitional*, porém permite *tags* específicas para o uso de *frames*. Estes *frames* eram utilizados no início da linguagem HTML para aninhar e apresentar páginas externas inteiras dentro de uma página ou como uma espécie de mosaico onde seriam encaixadas diversas páginas. Este recurso foi descontinuado nas versões mais atuais do HTML por gerarem diversos problemas de formatação e funcionalidades. Este recurso está em desuso, mas pode eventualmente ser encontrado em *sites* mais antigos (W3 XHTML, s.d.).

Glossário

Doctype: tipo de documento forma reduzida de se referir a DTD – "*Document Type Definition*".

DTD – *"Document Type Definition":* ou Definição do Tipo do Documento HTML ou XHTML.

Hipertexto: sistema de organização da informação, no qual certas palavras de um documento estão ligadas a outros documentos, exibindo o texto quando a palavra é selecionada.

Tags: ou etiquetas são os comandos de formatação da linguagem de marcação, chamados de elementos da linguagem. Um elemento é formado por um nome de etiqueta (*tag*), atributos e valores. Os atributos modificam os resultados padrões dos elementos e os valores caracterizam essa mudança.

Questão para reflexão

Você viu neste módulo as principais linguagens de marcação utilizadas na *web*, o HTML, o XML e o XHTML. Faça um breve relatório descrevendo as principais *tags* que pertencem exclusivamente a um documento XHTML e também quais as principais regras para a criação de *tags* personalizadas em XML. Isto enriquecerá o seu entendimento a respeito das semelhanças e diferenças entre estas linguagens.



Considerações Finais

Neste módulo, você aprendeu:

- Linguagem SGML – linguagem de marcação usada para grande quantidade de documentos.
- Linguagem HTML – conceitos iniciais da linguagem.
- Linguagem XML – linguagem de marcação mais flexível que proporcionava maior facilidade para novas tecnologias.
- Versões de HTML E XHTML.
    - O que é HTML – sua evolução e novas versões.
    - DTD – Document Type Definition – Definição do tipo de linguagem utilizada.
    - XHTML – A junção do HTML com as funcionalidades do XML.

Referências

COSTA, Carlos J. **Desenvolvimento para web**. Lisboa-Portugal: Edições Lusocrédito, 2007.

DICIO. Disponível em: <http://www.dicio.com.br/hipertexto/>. Acesso em: 11 set. 2016.

FRONTEND – FrontEnd Brasil. Disponível em: <http://www.frontendbrasil.com.br/artigos/a-historia-do-html/>. Acesso em: 11 set. 2016.

IBM – Tutorial XML. Disponível em: <http://www.ic.unicamp.br/~celio/inf533/docs/ibm_xml_tutorial.pdf>. Acesso em: 11 set. 2016

IBM K – IBM Knowledge Center. Disponível em: <http://www.ibm.com/support/knowledgecenter/SS6RBX_11.4.2/com.ibm.sa.xml.design.doc/topics/c_history.html>. Acesso em: 11 set. 2016.

PORTALEDUCAÇÃO. Disponível em: <http://www.portaleducacao.com.br/informatica/artigos/4475/o-que-e-xhtml>. Acesso em: 11 set. 2016.

W3 – Site do W3 Org. Disponível em: <https://www.w3.org/>. Acesso em: 11 jul. 2016.

W3 HTML. Disponível em: <https://www.w3.org/html/>. Acesso em: set. 2016.

W3 XHTML. Disponível em: <https://www.w3.org/MarkUp/?cm_mc_uid=22832131899614701448819&cm_mc_sid_50200000=1470150003>. Acesso em: 11 set. 2016.

Assista a suas aulas



Aula 2 - Tema: Indo além da linguagem de marcação. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/dc4aea8c3574bdd6ea0e51d6610e7669>.



Aula 2 - Tema: Indo além da linguagem de marcação. Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/ec2d9d89689d7474a38a72c0383e3019>.

Questão 1

1. Pode-se dizer que a evolução cronológica das linguagens de marcação foi:

a) SGML → HTML 1.0 → HTML xx → XML → HTML 4.01 → XHTML.

b) XML → HTML 1.0 → HTML xx → XGML → HTML 4.01 → XHTML.

c) SGML → XML 1.0 → HTML xx → HTML 2.0 → HTML 4.01 → XHTML.

d) SGML → HTML 1.0 → HTML 4.01 → HTML xx → XML → XHTML.

e) SGML → XML → XHTML → HTML 1.0 → HTML xx → HTML 4.01.

Questão 2

2. Assinale a alternativa correta. O Doctype é na prática:

a) Uma *tag* de abertura utilizada pelo HTML apenas.

b) Uma extensão de arquivo usada por arquivos *web*.

c) Definição de tipo de arquivo SGML.

d) Definição de tipo de arquivo XHTML.

e) Definição do tipo de linguagem utilizada para criar o documento *web* que será interpretado pelo navegador.

Questão 3

3. A linguagem SGML - “Standard Generalized Markup Language” foi desenvolvida:

a) Na década de 1980 pelo “IETF” (*Internet Engineering Task Force*).

b) Na década de 1960 pela IBM.

c) Na década de 1990 pela APPLE.

d) Na década de 1970 pelo FBI.

e) Na década de 1990 por Tim Berners Lee.

Questão 4

4. Assinale a alternativa correta. O termo XML, que é uma linguagem de marcação, significa:

a) *Express Markup Language.*

b) *Extensible Markup Link.*

c) *Extensible Markup Language.*

d) *Express Marker Link.*

e) *Extensible Marker Link.*

Questão 5

5. Pode-se afirmar que o XHTML é:

a) Uma linguagem de marcação baseada apenas em XML.

b) Uma linguagem de marcação baseada apenas em SGML.

c) Apenas uma versão mais moderna de HTML.

d) HTML com suporte a aparelhos de *smartphones* e *tablets*.

e) A junção das linguagens HTML e XML.

Gabarito

1. Resposta: A.

As demais opções trazem uma ordem cronológica incorreta, invertendo posições das linguagens em sua ordem.

2. Resposta: E.

Nas outras opções encontram-se definições incorretas, tais como o *DOCTYPE* ser apenas uma *tag* de abertura ou definição de apenas uma linguagem como o SGML ou XML.

3. Resposta: B.

As outras opções trazem informações inverídicas e fantasiosas sobre a data e o autor responsável pela criação da linguagem SGML.

4. Resposta: C.

As demais opções trazem termos inexistentes e fantasiosos que não representam a sigla em inglês XML.

5. Resposta: E.

Nas outras opções encontramos definições incorretas para o XHTML, como, por exemplo, ela ser baseada apenas em uma linguagem de marcação ou ser destinada apenas para tecnologias *mobiles*.

Unidade 3

Estrutura básica de um documento HTML e como podemos trabalhar com XHTML

Objetivos

Neste módulo, você aprofundará seus conhecimentos nas linguagens HTML e XHTML, suas estruturas básicas, como declarar corretamente o DTD/Doctype em cada caso e como isto poderá ser útil na construção de *sites*.

1. Estudar a estrutura básica de um documento HTML.
2. Ver as principais tags HTML.
3. Perceber como trabalhar com XHTML.

Introdução

Quando Tim Berners Lee inventou o **Protocolo HTTP** e a linguagem HTML em 1990, permitiu que a internet se tornasse possível, auxiliando diferentes tecnologias para a troca de conteúdos. O HTML, uma linguagem de marcação de hipertexto baseada em SGML, dava suporte para isso, sendo que inicialmente cada texto teria uma etiqueta (*tag*) que representaria um *link* entre dois documentos ou dois pontos de um mesmo documento, estabelecendo a ideia de hipertexto (FRONTEND, 2016).

Como você já teve oportunidade de ver, as linguagens de marcação tiveram uma evolução desde o surgimento da internet na seguinte ordem:

SGML → HTML 1.0 → HTML xx → XML → HTML 4.01 → XHTML.

Dessa forma, as linguagens de marcação atuais baseadas em HTML e XML mantêm estruturas e elementos básicos e, de certa forma, obrigatórios para que haja uma conformidade com as orientações de linguagem estabelecidas pelo W3C para que os *sites* possam ser corretamente interpretados pelos diversos tipos de navegadores e tecnologias de dispositivos que acessam a internet. Você verá quais estruturas são essas e aprofundará seus conhecimentos nas linguagens de marcação utilizadas.

1. Estrutura básica de um documento HTML

Quando os *sites* são acessados pelos navegadores de *internet*, é realizada uma transferência/**download** de um documento em

uma linguagem de marcação que será interpretada pelo navegador e mostrada ao usuário de uma forma legível, com textos formatados, imagens, vídeos e outros. Isso se tornou possível através do surgimento do HTML e atualmente a maioria das linguagens de marcação e tecnologias usadas na internet utilizam a estrutura do HTML para poderem funcionar e serem interpretadas pelos navegadores.

O HTML mantém uma estrutura mínima básica, como você pode observar:

```
<!DOCTYPE html>
<html >
<head>
<!-- Aqui vai o conteúdo do head usado pelo navegador -- >
<title> Título da página </title>
<meta charset=”utf-8”>
</head>
<body>
<!-- Aqui vai o código HTML que fará seu site aparecer -- >
</body>
</html>
```

Para que você possa compreender melhor esta **estrutura do código HTML**, podemos dividi-lo em três partes:

- ***Body:*** Corpo do Documento HTML.
- ***Doctype:*** Definição da linguagem e versão utilizada para construir o documento.
- ***Head:*** Cabeçalho do documento HTML.

2. Principais *tags* HTML

No código do exemplo acima, de estrutura mínima básica do HTML, temos uma divisão clara de três partes. Na primeira, o *doctype* é inserido antes da *tag* <HTML>. Isto serve para o navegador identificar qual é o tipo de HTML usado no documento que ele interpretará. Isto mostra se o documento foi escrito em HTML, XHTML, HTML 5 ou outra linguagem de marcação que possa ser interpretada pelo navegador.

A partir deste ponto, temos algumas *tags* que pontuam o documento em HTML e que também servem de base e padrão para outras linguagens (W3 HTML, 2016).

A primeira *tag* é a que inicia e encerra o documento, "<html> </html>". Entre estas duas tags a página a ser exibida deverá ser escrita. É comum quando se escreve um código que se faça a indentação de suas *tags* como mostrado no exemplo, ou seja, o alinhamento entre a *tag* que abre o elemento "< >" e a que o fecha "</ >". Isto gera uma maior clareza entre as seções do código e os respectivos comandos.

As duas seções principais de um código HTML são:

2.1 Head

No <head> </head> são colocadas as informações que serão usadas pelo navegador. Com exceção da *tag* "<title>", onde é colocado o título da página que será mostrado pelo navegador, as demais informa-

ções colocadas, nesta seção, serão usadas apenas pelo navegador. Nesta seção são colocadas instruções para o carregamento de páginas de estilo CSS, páginas externas e principalmente as **meta tags**, que trazem informações sobre o tipo de codificação de texto usado, descrição do conteúdo da página, informações de palavras-chaves para mecanismos de buscas, entre outros (INFOWESTER, 2016).

Para saber mais

Observe o exemplo de trecho de código em HTML. Aqui temos algumas *tags* utilizadas na seção "<head>":

```
<!doctype html>
<head>
<title>Título da página</title>
<meta charset="utf-8" />
<meta name="description" content="-
Descreve o conteúdo da página" />
<link href="estilo.css" rel="s-
tylesheet" type="text/css" />
</head>
```

Link

Você poderá ver uma descrição mais detalhada sobre HTML e as meta-tags em: <https://www.w3.org/TR/html401/struct/global.html#h-7.4.4>. Acesso em: 10 out. 2016.

2.2 Body

Na seção “<body> </body>” são colocadas as *tags* que irão compor todo o conteúdo do *site*, ou seja, tudo o que será mostrado pela página deverá estar escrito no <body>. Faz parte deste conteúdo os textos apresentados, as imagens, sons, vídeos, músicas e todos os aplicativos embarcados, como serviços de mapas, geoposicionamento GPS, além de todas as entradas de dados como senhas, formulários e conexões com páginas externas que possam retornar dados e apresentá-los ao usuário.

As *tags* em HTML, na prática, trazem os elementos da linguagem, ou seja, os comandos que serão interpretados pelos navegadores e formarão as páginas da internet (W3 HTML, 2016).

Existem diversas *tags* na linguagem HTML, porém seguem uma regra de sintaxe semelhante. Veja o exemplo:

```
1|  <center><h1>EXEMPLO DE TAGS HTML</h1></center>
2| <br>
3|  <p><center><font color=”blue”
```

```
face="Arial" size="7"> Este é um texto simples em Azul com fonte Arial tamanho 7 </font> </center> </p>
```

Observando o código do exemplo, na linha 1 temos a *tag* <h1>, que é utilizada para marcar um título. Como este código está escrito em HTML, os elementos de alinhamento e formatação dos textos estão escritos diretamente no código. A *tag* <Center> faz o alinhamento desta linha de texto.

Na linha 2, a *tag*
 pula uma linha. Já na linha 3 temos a *tag* <p>, que serve para descrever um parágrafo de texto. Observe que esta linha tem algumas *tags* alinhadas que servem de parâmetros de formatação. No caso da *tag* "<font color="blue" face="Arial" size="7">" temos a seguinte organização: (<font) é a *tag*; (color) é o atributo; (="blue") é o valor do atributo (color) e deve ser colocado sempre entre "aspas". O mesmo acontece para os atributos (face) e (size) que recebem seus valores entre "aspas". Em geral, as ***tags*** tem seus elementos, os elementos têm seus atributos, estes atributos recebem seus respectivos valores que devem ser colocados entre "aspas" (W3 HTML, 2016).

Link

Você poderá ver uma descrição completa das *tags* HTML e seus parâmetros em: <https://developer.mozilla.org/pt-BR/docs/Web/HTML/Element>. Acesso em: 10 out. 2016.

Lembre-se de que o HTML é uma linguagem de marcação de hipertexto e não precisa de um compilador para mostrar o seu resultado. Esta é interpretada diretamente pelo navegador.

Para saber mais

Um documento HTML pode ser criado através de qualquer editor de texto, porém deve ser salvo com a extensão ".html" ou ".htm". Os documentos em HTML só serão interpretados pelos navegadores se estiverem gravados em uma dessas duas extensões.

3. Como trabalhar com XHTML

A linguagem XHTML do inglês "*Extensible HyperText Markup Language*" apresenta uma tradução livre de "Linguagem de marcação de hipertexto extensível". Como você pode perceber, o XHTML é a junção do HTML e sua estrutura apresenta flexibilidade e diversas funcionalidades do XML, ou seja, ele combina as *tags* do HTML com as regras do XML, o que amplia o HTML e o reformula em XML.

O XHTML foi pensado para proporcionar uma maior compatibilidade das aplicações *web* e suas novas tecnologias. Uma característica que torna o uso do XHTML mais eficiente é o seu carregamento rápido pelos navegadores devido ao seu código ser mais limpo e às recomendações dadas a ele pelo W3C (W3 XHTML, 2016).

A linguagem XHTML é uma recomenda-

ção do W3C publicada em 26 de janeiro de 2000, tornando-a uma *Web Standard*.

O XHTML é baseado no HTML, mas tem algumas diferenças em relação a ele, por exemplo:

- Todas as *tags* devem ser escritas em letras minúsculas, pois o XHTML assim como o XML é *"case sensitive"*, ou seja, sensível a letras maiúsculas ou minúsculas.
- As *tags* devem estar corretamente aninhadas, ou seja, numa ordem correta de abertura e fechamento.
- As *tags* devem estar corretamente colocadas na estrutura básica do HTML.
- É necessário usar sempre *tags* de fechamento.
- Os atributos de um elemento também devem estar em letras minúsculas.

Um ponto a ser destacado refere-se às declarações do tipo de documento. Como já visto, para que o **XHTML** possa suportar as tecnologias anteriores a ele em HTML, estabeleceu-se o uso de três tipos de "*Doctype*", que são respectivamente declarados dessa forma (W3 XHTML, s.d.):

- **Strict:**

```
<!DOCTYPE html
    PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

- **Transitional:**

```
<!DOCTYPE html
PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

- **Frameset:**

```
<!DOCTYPE html
    PUBLIC "-//W3C//DTD XHTML 1.0 Frameset//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-frameset.dtd">
```

Para saber mais

Da mesma forma que um documento HTML, o documento XHTML pode ser criado através de qualquer editor de texto e deve ser salvo com a extensão “.html” ou “.htm”. Ao interpretar o documento, o navegador irá identificar que ele está codificado em XHTM através da descrição contida no “DOCTYPE”.

Link

Você pode encontrar uma descrição completa do XHTML e suas recomendações publicadas pelo W3C já traduzido em português em: <http://www.maujor.com/w3c/xhtml10_2ed.html>. Acesso em: 10 out. 2016.

Glossário

Download: transferir (baixar) um ou mais arquivos de um servidor remoto para um computador local (SIGNIFICADOS).

Meta tags: meta tags são linhas de código HTML ou "etiquetas" que, entre outras coisas, descrevem o conteúdo do seu *site* para os buscadores da internet.

Protocolo HTTP: sigla do inglês "HyperText Transfer Protocol" ou numa tradução livre "Protocolo de Transferência de Hipertexto" permite a comunicação entre sistemas de diferentes tecnologias e a transferência de dados entre redes de computadores. Este protocolo é usado como base pela *World Wide Web* – Internet.

Questão para reflexão

Você viu neste módulo as principais *tags* das linguagens HTML e XHTML. Para que você possa compreender melhor estas linguagens e suas diferenças, procure desenvolver no bloco de notas duas páginas que contenham os seguintes itens:

- Na página 1: Definição de DOCTYPE, título da página, um texto centralizado na página, duas linhas em branco, uma imagem .jpg de sua escolha centralizada, novamente duas linhas em branco, e finalizando um texto também centralizado que será o *link* para a página 2.
- Na página 2: os mesmos itens, porém com uma imagem .jpg diferente e finalizando um texto também centralizado que será o link para a página 1.



Questão para reflexão



Para que os *links* possam funcionar corretamente, coloque os dois arquivos na mesma pasta, não se esquecendo de gravá-los com a extensão “.html” ou “.htm”. Você poderá testar a qualquer momento sua criação abrindo seu documento com um navegador *web*.

Considerações Finais

Neste módulo, você aprendeu os assuntos relacionados à(a):

- Estrutura Básica de um documento HTML – Uma visão da estrutura básica e suas principais seções, declaração do Doctype HTML, as tags de marcação <html>, <head> e <body>
- Principais tags HTML – Aprofundamento das seções da estrutura básica e as principais tags utilizadas.
- Como trabalhar com XHTML – As principais diferenças entre o XHTML e o HTML e como deve ser escrito corretamente o XHTML e declarados os "*DOC-TYPES*" para cada caso desta linguagem.

Referências

FRONTEND. Site FrontEnd Brasil. Disponível em: <http://www.frontendbrasil.com.br/artigos/a--historia-do-html/>. Acesso em: 10 out. 2016.

INFOWESTER. Site Infowester. Disponível em: <http://www.infowester.com/metatags.php>. Acesso em: 10 out. 2016.

SIGNIFICADOS. Site Significados. Disponível em: <http://www.significados.com.br/>. Acesso em: 10 out. 2016.

W3. Site do W3 Org. Disponível em: <https://www.w3.org/>. Acesso em: 10 out. 2016.

W3 HTML. Disponível em: <https://www.w3.org/html/>. Acesso em: 10 out. 2016.

W3 XHTML. Disponível em: <https://www.w3.org/MarkUp/?cm_mc_uid=22832131899614701448819&cm_mc_sid_50200000=1470150003>. Acesso: 10 out. 2016.

Assista a suas aulas



Aula 3 - Tema: Estrutura básica de um documento HTML e como podemos trabalhar com XHTML. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/b81636dab9164b49108bccb3afdfe605>.



Aula 3 - Tema: Estrutura básica de um documento HTML e como podemos trabalhar com XHTML. Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/c07b91fcded36676e5edc8a26935c982>.

Questão 1

1. Assinale a alternativa correta. Um documento em XHTML pode ser gravado com a extensão:

a) ".xhtm" ou ".xhtml".

b) ".html" ou ".htm".

c) ".xml" ou ".xhtm".

d) ".html" ou ".xhtm".

e) ".htmlx" e ".htmx".

Questão 2

2. Assinale a alternativa correta. Na linguagem XHTML temos diferentes "*DOCTYPES*" que identificam os recursos aceitos pela linguagem. São eles:

a) Strict, Transitional e Blink.

b) XHTML, Transitional e Frameset.

c) Strict, Transitional e XML.

d) Strict, Transitional e Frameset.

e) #Strict, #XML e #Frameset.

Questão 3

3. Na codificação de um documento HTML, na seção que compreende as *tags* <head> </head> são colocadas:

a) *Tags* utilizadas apenas pelo navegador e metadados de informações da página. Estas informações não são mostradas no *site* com exceção do título.

b) *Tags* utilizadas apenas pelo navegador e metadados de informações da página. Estas informações são sempre mostradas em algum ponto do *site*.

c) Informações que serão mostradas no site, como textos, imagens, vídeos e outros.

d) Apenas o *DOCTYPE* do documento.

e) Apenas o título da página.

Questão 4

4. *Tags* sempre em letras minúsculas, necessidade de *tags* de fechamento e atributos também sempre em letras minúsculas são recomendações para a linguagem:

a) HTML e XHTML.

b) HTML 4.01 apenas.

c) XHTML apenas.

d) XGML apenas.

e) HTML 4.0 e suas versões anteriores.

Questão 5

5. Na codificação de um documento HTML, na seção que compreende as *tags* <body> </body> são colocadas:

a) *Tags* utilizadas apenas pelo navegador e metadados de informações da página. Estas informações são sempre mostradas em algum ponto do *site*.

b) *Tags* de formatação de textos e imagens do *site* apenas.

c) Apenas o *DOCTYPE* do documento.

d) *Tags* para chamadas de conteúdos externos que devam ser aplicados ao *DOCTYPE* do documento.

e) *Tags* que irão compor todo o conteúdo do *site*, como textos, imagens, sons, vídeos, formulários de entrada, entre outros.

Gabarito

1. Resposta: B.

Um documento XHTML é identificado pelo navegador da mesma forma que o documento HTML quando este está gravado com a extensão ".html" ou ".htm" apenas. Em todas as outras opções foram usados tipos de extensão que sequer existem.

2. Resposta: D.

Nas demais opções temos itens que não são definições de documento, chamados de DOCTYPES, como o Blink, XHTML, XML, ou que sequer existem, como o caso de #XML e #Frameset.

3. Resposta: A.

As outras opções descrevem de forma incorreta as informações do <head> e como são mostradas pelo navegador, contendo até descrições que não pertencem a esta seção, como é o caso do *DOCTYPE*.

4. Resposta: C.

Nas demais questões são citadas linguagens de marcação com recomendações diferentes em relação ao uso de maiúsculas e minúsculas em suas *tags*, como o caso do HTML que não faz esta distinção em seu código, ou seja, aceita da mesma forma maiúsculas e minúsculas.

5. Resposta: E.

Gabarito

As outras opções descrevem de forma incorreta as informações do <body> e como são mostradas pelo navegador, contendo até descrições que não pertencem a esta seção, como o *DOCTYPE* e os metadados.

Unidade 4

Trabalhando com CSS

Objetivos

A internet e os milhões de *sites* que a compõem absorvem cada vez mais novos recursos e tecnologias. Neste contexto, o HTML e as demais linguagens de marcação precisaram evoluir para poder atender a estas novas possibilidades, e o surgimento bem como o uso de CSS tornou-se fundamental para que isto acontecesse. Você verá:

1. O que é o CSS.
2. Estrutura do CSS.
3. Gramáticas e Convenções do CSS.
4. Box Model e Responsividade na *Web*.

Introdução

Inicialmente inventado para funcionar com hipertexto, o HTML proporciona a ligação entre dois documentos diferentes ou dois pontos dentro do mesmo documento, além de ser uma forma de padronização de apresentação destes documentos através de um navegador (*browser*). Com o passar do tempo novos recursos visuais surgiram, as páginas começaram a apresentar diversos padrões de cores, tipos de letras, conteúdos divididos em colunas, caixas de texto e vários outros recursos de *design* de páginas que não eram totalmente suportados pelo HTML inicial. Uma solução foi adicionar novas "*tags*" à linguagem, mas com o tempo isto acabou gerando novos problemas de desenvolvimento. Os "***web designers***" começaram a criar páginas cada vez mais carregadas de recursos visuais, estilos de fontes e tipos extremamente variados. O problema geralmente surgia na hora de uma alteração ou modificação visual, por exemplo, para mudar o tipo da letra de um título de uma página este trabalho seria simples, mas se tivéssemos que mudar o tipo de letra de um *site* inteiro com 200 páginas isto seria extremamente trabalhoso. Isto acontecia porque os estilos e definições visuais estavam todos inseridos dentro da estrutura da linguagem HTML. Pensando nisso, foi criada uma nova maneira de formatar este conteúdo, o CSS.

1. O que é o CSS

Em 1995, Hakon Wium Lie e Bert Bos apresentaram a proposta do CSS, sigla em inglês

para *"**Cascading Style Sheets**"*, ou em uma tradução livre, Páginas de Estilo em Cascata, que era uma solução apresentada para tirar da estrutura básica de uma página HTML os componentes de formatação visual de um *site*. Esta ideia foi apoiada pela W3C e rapidamente passou a ser uma recomendação do W3C.

Basicamente, a ideia consiste em utilizar o HTML somente para estruturar o *website* e a sua formatação visual é feita com o CSS, onde se coloca em um arquivo separado com extensão ".css" as "*tags*" de formatação. Etilos de fonte, tamanho, cor, elementos visuais como marcação de títulos, subtítulos, fundos da página e outros são descritos neste arquivo CSS. Ao invés de colocar a formatação dentro do código HTML, o desenvolvedor cria um "*link*" no código "chamando" o arquivo externo CSS – ".css".

É possível também colocar estes "estilos" e suas "*tags*" dentro do arquivo HTML, inserindo o trecho de **CSS** dentro do código da página. Este recurso é indicado quando é necessário formatar apenas a página em questão, porém, o CSS é melhor aplicado quando os elementos de formatação são colocados em um arquivo separado ".css" para que eles possam ser aplicados às diversas páginas de um *site*.

Link

Você poderá ver uma descrição mais aprofundada sobre HTML com CSS em: <https://developer.mozilla.org/en-US/docs/Web/CSS>. Acesso em: 10 out. 2016.

2. Estrutura do CSS

2.1 Criando uma folha de estilo CSS e chamando no código HTML

Para poder criar uma **folha de estilo** CSS você deve primeiro criar dois arquivos dentro da pasta que receberá os arquivos do *site*. Nela, você deve gravar os arquivos "index.html" que será a sua página inicial e o arquivo "estilo.css", onde serão gravados as *tags* CSS.

Para saber mais

Observe o exemplo a seguir que descreve como chamar uma folha de estilo CSS dentro da página HTML.

Abra o bloco de notas e digite o código:

```
strong {color: blue;}
```

Salve o arquivo como "estilo.css".

Para saber mais

Abra um novo arquivo no bloco de notas e digite o código:

```
<DOCTYPE html>
<html>
 <head>
    <meta charset="UTF-8">
    <title>Exemplo de CSS</title>
    <link rel="stylesheet" hre-
    f="estilo.css">
 </head>
 <body>
    <p> conteúdos do site </p>
 </body>
<html>
```

Salve o arquivo como "index.html".

Note que a *tag* "`<link rel="stylesheet" href="estilo.css">`" faz a ligação entre o documento HTML com a folha de estilo CSS que está no arquivo externo. Esta *tag* deve ser colocada na seção <head> do HTML chamando assim o CSS para a página e aplicando os estilos descritos nos elementos HTML como textos, cores e outros.

2.2 Gramáticas e convenções do CSS

Quando o navegador exibe o documento HTML que chama uma página de estilo CSS, ele deve combinar o conteúdo deste documento com as informações de estilo definidas no arquivo chamado. O navegador processa essas informações em duas etapas:

- O navegador converte os comandos de HTML e de CSS para o "*DOM* " ("*Document Object Model*"), que funciona como um arquivo temporário de trabalho para que HTML e CSS possam funcionar juntos, combinando o as "tags" HTML com seus estilos. Ao final, o navegador exibe o conteúdo do DOM. Entendendo o DOM ajuda você a criar, depurar e manter o seu CSS claro e objetivo, pois o DOM é onde o seu CSS e conteúdo do documento se encontram.
- Ao final, o navegador exibe o conteúdo do DOM. Entender o DOM ajuda você a criar, depurar e manter o seu CSS claro e objetivo, pois o DOM é onde o seu CSS e conteúdo do documento se encontram.

3. Cascata e herança em CSS

O estilo que um elemento HTML terá após

Link

Você encontra uma descrição detalhada sobre "*DOM – Document Object Model*" (em inglês, que pode ser traduzido pelo *plugin* do Google Chrome), em: <https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model>. Acesso em: 10 out. 2016.

o navegador interpretar o código pode ser especificado em vários locais diferentes, e que podem interagir com estes elementos de uma forma complexa. Esta complexidade faz o CSS poderoso, mas também torná-lo confuso e difícil de depurar.

Três possibilidades de definição do estilo formam uma *cascata de estilos:*

- Estilos-padrão do navegador – por exemplo tipo de fontes, tamanho, se normal ou negrito, alinhamento, cor, entre outros.
- Estilos definidos no navegador do usuário que está lendo o documento.
- Os estilos definidos pelo autor do documento.

Voltando ao nosso exemplo de código com CSS:

`strong {color: blue;}` – Conteúdo do arquivo "estilo.css".

E temos também o código da página "index.html":

```
<!DOCTYPE html>
<html>
    <head>
      <meta charset="UTF-8">
      <title>Exemplo de CSS</
      title>
      <link rel="stylesheet"
      href="estilo.css">
    </head>
    <body>
      <p><strong> conteúdos do
      site </strong></p>
      <p> sou um texto inserido na
      página sem strong </p>
```

```
    </body>
<html>
```

Quando o navegador interpretar este código, o texto definido no parágrafo   como será mostrado em azul, pois está definido na página de estilo CSS, mesmo que a **cor padrão** para textos no navegador esteja definida como "preto" por exemplo. Isto forma uma hierarquia que é seguida pelos navegadores, onde as definições da folha de estilo do autor têm prioridade, em seguida vem os padrões definidos pelo navegador. No exemplo acima a "tag" " sou um texto inserido na página sem strong " recebe as definições do navegador, pois nesta linha não foi inserida a definição "strong" da folha de estilo.

Para saber mais

Experimente algumas variações utilizando este mesmo código. Vá até o código CSS e modifique a cor de para yelllow, salve o arquivo e abra novamente sua página HTML. Você verá que a cor foi atualizada para o novo parâmetro. Esta é a ideia de se trabalhar com CSS. As configurações visuais são definidas no código CSS e automaticamente atualizam os elementos do código HTML.

Link

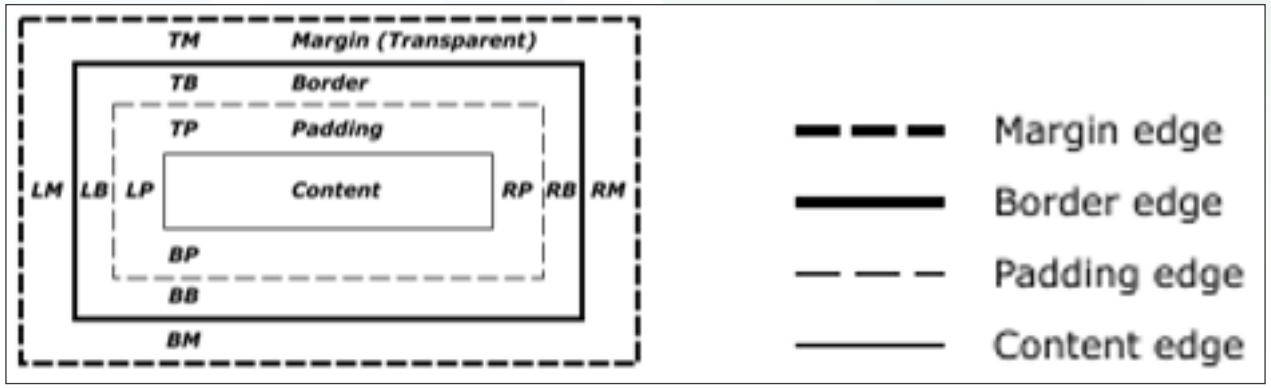
Você encontra uma um tutorial completo sobre como utilizar corretamente CSS em: <https://www.w3.org/Style/Examples/011/firstcss>. Acesso em: 10 out. 2016.

4. Box model e responsividade na *web*

Em uma página *WEB*, cada elemento é representado como um quadro retangular, chamado também de "**BOX**". Determinar o tamanho, suas propriedades como cor, fundo, estilo de bordas e a posição desses quadros é o objetivo dos motores de renderização.

No CSS, cada um desses *quadros* é descrito usando o "*box model*" *(quadro modelo)* padrão. Este modelo descreve o conteúdo do espaço ocupado por um elemento. Cada "Box" possui quatro "edges" ou bordas possíveis: **margin edge**, **border edge**, **padding edge** e **content edge.**

Figura 1 | Descrição das bordas possível em um “Box Model”



Fonte: <https://www.w3.org/TR/CSS2/box.html>. Acesso em: 10 out. 2016.

- **Área de conteúdo (content area)** é a área ocupada pelo conteúdo real do elemento.
- **Área de preenchimento (padding area)** e a borda em torno do conteúdo.
- **Área de borda (border area)**
- **Área de margem (margin area)** utilizado para separar o box dos elementos vizinhos.

Para saber mais

Os recursos que tornam os *sites* e suas páginas autoajustáveis aos diversos tamanhos e tipos de monitores e dispositivos são chamados de responsividade ou *tableless*. Estes recursos só são possíveis com a utilização dos Box Models.

Glossário

Box: no contexto de Box Model em CSS é essencialmente uma caixa que envolve todos os elementos HTML.

Cascading Style Sheet: em uma tradução livre “páginas de estilo em cascata”, recurso do HTML para a formatação visual dos elementos de uma página.

Web designers: profissional responsável pela concepção da arquitetura de um *site* na internet, incluindo a sua estrutura e o aspecto das diversas páginas que o compõem.

Questão para reflexão

Aproveitando os conceitos vistos neste módulo, procure criar uma página que contenha os seguintes itens:

- Definição de DOCTYPE, título da página definido como “Minha Primeira Página Pessoal”.
- Um Box model com suas respectivas definições de margens.
- Dentro do Box Model um texto de apresentação pessoal contendo Título em letra Times New Roman, tamanho 14, Bold, centralizado, cor Azul e pulando duas linhas um parágrafo de texto com sua apresentação pessoal em Times New Roman, 12, normal, cor.
- Um novo Box Model logo abaixo, contendo uma imagem .jpg (sua foto) de sua escolha, centralizada.
- Os itens acima devem ser descritos através de um arquivo .CSS externo e ser chamado nas páginas HTML.



Considerações Finais

Neste módulo, você aprendeu:

- O que é o CSS.
- Estrutura do CSS.
- Gramáticas e Convenções do CSS.
- Box Model e Responsividade na *Web*.

Referências

MOZILA. Site disponível em: <https://developer.mozilla.org/>. Acesso em: 10 out. 2016.

SIGNIFICADOS. Site Significados. Disponível em: <http://www.significados.com.br/ >. Acesso em: 10 out. 2016.

TABLELESS. Disponível em: <http://tableless.com.br/entendendo-sistemas-de-grid-css-do-zero/>. Acesso em: 10 out. 2016.

W3 – Site do W3 Org. Disponível em: <https://www.w3.org/>. Acesso em: 10 out. 2016.

w3schools. Site. Disponível em: <http://www.w3schools.com/css/css_boxmodel.asp>. Acesso em: 10 out. 2016.

Assista a suas aulas



Aula 4 - Tema: Trabalhando com CSS. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f-1d/0e1a2e6cb0ee1c470edf9882036bafe5>.



Aula 4 - Tema: Trabalhando com CSS. Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f-1d/08e15f781a3b2b7d28f38fa4450af352>.

Questão 1

1. Assinale a alternativa correta. O "DOM " ("*Document Object Model*"), em uma página HTML que utiliza os recursos CSS, tem a função de:

a) São os comandos CSS que são processados pelo navegador.

b) Funciona como um arquivo temporário que une o código HTML com o código CSS e exibe o resultado final como página de internet.

c) É o código que executa a parte visual do CSS no navegador após a execução do HTML.

d) É a codificação gerada pelo CSS para que possam ser usadas linguagens orientadas a objetos.

e) São os objetos gerados pelo CSS na página do site construída com a linguagem HTML.

Questão 2

2. Assinale a alternativa correta. O CSS sigla em inglês para "*Cascading Style Sheets*", consiste basicamente em:

a) Formatação visual dos textos e das fontes de letra de um *site*.

b) Formatação visual de fundos de imagens e cores de fundo de um *site*.

c) Código complementar ao HTML que ajusta o visual do *site* na tela que estiver sendo apresentado o *site*.

d) Formatação visual de todos os elementos de um site através de páginas de estilo.

e) Um recurso de formatação, dividindo o código HTML em seções de acordo com a sua função no *site*.

Questão 3

3. Para criar uma folha de estilo CSS, a maneira mais correta e usual é:

a) Devem-se colocar as tags CSS dentro da seção <HEAD>, não sendo mais necessário tipo algum de outro caminho.

b) Devem-se descrever as *tags* de comando CSS em conjunto com cada uma das tags de comandos HTML.

c) Criar dois arquivos dentro da pasta do *site*, um arquivo HTML e outro arquivo "estilo.css" onde serão gravados os comandos, as *tags* CSS.

d) O código CSS funciona como um complemento da linguagem HTML e deve-se criar a seção <CSS> dentro da seção <HEAD> no código HTML.

e) O código CSS funciona como um complemento da linguagem HTML e deve-se criar a seção <CSS> dentro da seção <BODY> no código HTML.

Questão 4

4. Quando um navegador interpreta um código CSS, é definida uma hierarquia entre a definição contida no navegador e a contida no código CSS. Esta hierarquia prevê:

a) As definições da folha de estilo do autor têm prioridade, em seguida vêm os padrões definidos pelo navegador.

b) As definições de padrão do navegador têm prioridade, em seguida vêm os padrões definidos pela folha de estilo CSS.

c) As definições da folha de estilo do autor têm prioridade, caso não sejam definidas, os textos são definidos na fonte “courier new”, normal, tamanho 10.

d) As definições da folha de estilo do autor têm prioridade, caso não sejam definidas, os textos são definidos na fonte “times new roman”, normal, tamanho 11;

e) As definições da folha de estilo do autor têm prioridade, caso não sejam definidas, os textos são definidos na fonte “arial”, normal, tamanho 10.

Questão 5

5. O "Box Model", um recurso das páginas de estilo CSS, descreve o conteúdo do espaço ocupado por um elemento. Cada Box possui quatro bordas possíveis, que são:

a) top edge, base edge, left edge e rigth edge.

b) inside edge, outside edge, padding edge e content edge.

c) margin edge, border edge, inside edge e element edge.

d) margin edge, border edge, cell edge e colun edge.

e) margin edge, border edge, padding edge e content edge.

Gabarito

1. Resposta: B.

O navegador converte os comandos de HTML e de CSS para o "DOM " ("*Document Object Model*"), que funciona como um arquivo temporário de trabalho para que HTML e CSS possam funcionar juntos, combinando o as "tags" HTML com seus estilos. Ao final, o navegador exibe o conteúdo do DOM.

2. Resposta: D.

Basicamente a ideia consiste em utilizar o HTML somente para estruturar o *website* e a sua formatação visual é feita com o CSS, onde se coloca em um arquivo separado com extensão ".css" as "*tags*" de formatação. Ao invés de colocar a formatação dentro do código HTML, o desenvolvedor cria um "*link*" no código chamando o arquivo externo CSS – ".css".

3. Resposta: C.

Para poder criar uma folha de estilo CSS você deve primeiro criar dois arquivos dentro da pasta que receberá os arquivos do *site*. Nela, você deve gravar os arquivos "index.html" que será a sua página inicial e o arquivo "estilo.css", onde serão gravados as *tags* CSS.

4. Resposta: A.

No caso do uso de folhas de estilo, as definições da folha de estilo do autor têm prioridade, em seguida vêm os padrões definidos

pelo navegador.

5. Resposta: E.

Cada “Box” possui 4 “edges” ou bordas possíveis: margin edge, border edge, padding edge e content edge.

Unidade 5

Desvendando o universo Javascript

Objetivos

Entre as diversas linguagens existentes para o desenvolvimento de *sites* e tecnologias para a internet, temos a Linguagem Javascript. A linguagem Javascript é uma ferramenta no desenvolvimento de *sites*. Neste módulo, abordaremos os seguintes tópicos:

1. O que é Javascript.
2. Tipo de dados e variáveis.
3. Operadores, expressões e comandos em Javascript.
4. Documento *Object Model*.
5. Apresentação de bibliotecas e *frameworks*.

Introdução

O Javascript foi considerado um *script* de página e até como uma linguagem que servia apenas para acomodar novos recursos às páginas *web*. Na verdade, o Javascript é uma linguagem de *script* orientada a objetos. É necessário ressaltar que Javascript e Java são linguagens distintas, ou seja, embora JavaScript e Java são similares em alguns aspectos, são diferentes em outros (MOZILA, s.d.).

1. O que é Javascript

O Javascript assemelha-se ao Java, usa-se uma sintaxe básica do Java, com nomenclatura e construções de controle de fluxo, porém não é uma linguagem minimalista do próprio Java. A diferença entre as duas linguagens é que Java é uma linguagem compilada e interpretada por sua Máquina Virtual, a **JVM**, que proporciona o funcionamento de programas desenvolvidos em Java. Já a linguagem Javascript é um *script* de página, considera-se uma "linguagem do lado cliente", é interpretada diretamente pelo navegador e é usada para a inserção de elementos na página *web*, como formulários, senhas, avisos, janelas, cálculos e tantos outros criados diretamente com o **Javascript**. (ORACLE, s.d.).

Para saber mais

O Javascript mantém estruturas semelhantes ao Java e outras linguagens orientadas, porém é uma linguagem interpretada diretamente pelos navegadores, não havendo para isso necessidade de compiladores para esta linguagem.

O **JavaScript** é uma linguagem com regras de sintaxe mais dinâmicas e flexíveis, como o HyperTalk e **Dbase**.



Link

Você poderá ver uma descrição mais aprofundada sobre Javascript em: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Introduction#O_que_é_JavaScript>. Acesso em: 10 out. 2016.

2. Declarações, tipo de dados e variáveis

2.1. Sintaxe básica

O JavaScript aproveita algumas descrições de comando de outras linguagens, como o Perl e **Python**. O JavaScript é "***case-sensitive***" e as instruções são separadas por um ponto e vírgula (;).

O Javascript pode ser inserido de duas formas distintas em uma página *web*:

- Interno e diretamente no Documento HTML, como pode ser visto no exemplo do código a seguir:

```
<head>
    <script type="text/
javascript">
      //código Javascript
    </script>
  </head>
```

- Chamando o código Javascript através de um arquivo externo, como segue:

```
<head>
    <script type="text/javascript"
src="ArquivoemJavascript.js"></
script>
  </head>
```

Neste caso, o arquivo deve ser salvo com a extensão ".JS" para que possa ser chamado e interpretado corretamente pelo navegador.

2.2. Declarações

As variáveis são temporárias, servem para acumular informações e resultados de algum processamento, como cálculos por exemplo. É necessário antes de tudo **declarar** as variáveis que serão usadas.

Existem três tipos de declarações em JavaScript:

- **var** - Declara uma variável, opcionalmente, inicializando-a com um valor.
- **let** - Declara uma variável local de escopo do bloco, opcionalmente, inicializando-a com um valor.
- **const** - Declara uma constante apenas de leitura.

Veja um exemplo de declaração de **variá-**

veis:

```
var autor;
autor = "Tim Berners Lee";
var idade = 60;
```



Link

Você poderá encontrar a descrição mais aprofundada sobre como declarar estas variáveis em Javascript em: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/>.

Acesso em: 10 set. 2016.

Para saber mais

Uma variável é um objeto (uma posição, frequentemente localizada na memória) capaz de reter e representar um valor ou expressão. Enquanto as variáveis só "existem" em tempo de execução, elas são associadas a "nomes", chamados identificadores, durante o tempo de desenvolvimento.

3. Controle de fluxo

O Javascript oferece laços condicionais que se iniciam quando uma condição de comparação é colocada, enquanto esta condição não for atendida, a execução do código vai permanecer processando as informações contidas dentro do laço. O Javascript

tem comando de laço semelhantes aos de diversas outras linguagens, a seguir serão apresentadas as principais.

3.1. Do.... While

```
Do declaração
while (condição);
```

- **declaração**

A declaração é executada pelo menos uma vez e reexecutada cada vez que a condição for avaliada como verdadeira (*true*).

- **condição**

Uma expressão é validada depois de cada passagem pelo laço. Se a condição é avaliada como verdadeira (*true*) o bloco de código é executado novamente. Quando a condição é avaliada como falsa (f*alse*), o controle passa para a instrução seguinte ao laço **do... while**, ou seja, o programa sai do laço e segue em frente.

3.2. While

A d**eclaração while** cria um laço que executa uma rotina específica enquanto a condição de teste for avaliada como verdadeira. A condição é avaliada antes da execução da rotina.

```
while (condição) {
    rotina
}
```

- **Condição**

Uma expressão avaliada antes de cada passagem através do laço. Se essa condição for verdadeira, a rotina é executada. Quando a condição for falsa, a execução continua na declaração depois do laço while.

- **Rotina**

Uma declaração que é executada enquanto a condição é avaliada como verdadeira.

3.3. FOR

A instrução **FOR** cria um *loop* que consiste em três expressões opcionais, dentro de parênteses e separadas por ponto e vírgula, seguidas por uma declaração ou uma sequência de declarações executadas seguidamente.

```
for ([inicialização]; [condição]; [expressão final])
    Declaração
```

- **inicialização**

As variáveis são não locais no *loop*, isto é, elas estão no mesmo escopo que o loop **for** está. O resultado desta expressão é descartado.

- **condição**

Se a expressão for avaliada como falsa, a execução salta para a primeira expressão na sequência da construção.

- **expressão final**

Uma expressão para ser avaliada no final de cada iteração. Isto ocorre antes da próxima avaliação da condição. Geralmente usado para atualizar ou incrementar a variável do contador.

- **declaração**

Uma declaração que é executada enquanto a condição for avaliada como true.

Para saber mais

Um comando de Laço atende a uma condição Boolena "Verdadeiro ou Falso" (True / False). Por exemplo, enquanto determinada condição for False, executa-se o laço, e ao se atingir a condição de True, termina o laço e segue o código.

4. Trabalhando com funções

Funções são usadas para criar pequenos trechos de códigos separados do *script* principal. Estas funções podem gerar um resultado, chamadas de funções com retorno, ou apenas executar uma rotina, chamadas de funções sem retorno.

Funções também são úteis para serem facilmente chamadas, por exemplo como submeter um formulário, criar um Box de texto, entre outras.

Toda função tem um nome e possui um delimitador de parâmetro que é identificado com parênteses.

A declaração **function** define uma função com os parâmetros especificados.

Você pode também definir funções usando o “Construtor Function” e uma “function expression” (MOZILA, s.d.).



Link

Você poderá encontrar a descrição mais aprofundada sobre Funções em Javascript em: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Function>. Acesso em: 10 out. 2016.

5. Object model

O CSS Object Model é um conjunto de APIs que permite manipular o CSS através do JavaScript. Isto depende do DOM e da HTML APIs. Com isto, é permitido a leitura e a modificação dos estilos CSS dinamicamente. Este recurso é utilizado para a formatação visual das páginas de um *site* usando-se o Javascript.

6. API

A API é um conjunto de regras para realizar uma tarefa. Em alguns casos, uma API pode documentar alguns detalhes de como deve ser a implementação para garantir que alguns objetivos sejam alcançados.

6.1 Biblioteca

A biblioteca normalmente é uma implementação real das regras de uma API, costuma ser autossuficiente e pode conter implementações públicas que não fazem parte da API.

Você chama uma biblioteca. É uma ferramenta. Você usa o que precisa.

Pode-se ter uma biblioteca de *strings*, de expressões regulares, de coleções de dados, de manipulação de arquivos, acesso a um banco de dados, manipulação de imagens etc.

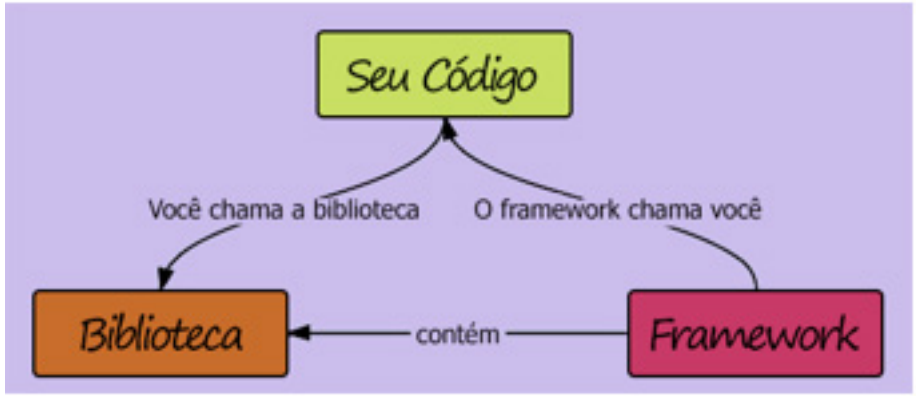
6.2 Framework

Um framework normalmente é um conjunto de bibliotecas para conseguir executar uma operação maior. Encapsula os comportamentos da API em implementações mais complexas, tornando o uso flexível.

É usado para determinar um conjunto de arquivos de bibliotecas de códigos binários, ou fontes ou até outros recursos para a aplicação, sendo que uma biblioteca quando se torna complexa, vira um framework.

Os toolkits, que podem ser confundidos com frameworks, mas funcionam de forma mais livre, são como bibliotecas funcionando em conjunto, mas de uma maneira tradicional, onde você "chama" o que precisa, conforme ilustra a Figura 2.

Figura 2 | Biblioteca x framework



Fonte: <http://pt.stackoverflow.com/questions/17501/qual-%C3%A9-a-diferen%C3%A7a-de-api-biblioteca-e-framework>. Acesso em: 10 out. 2016.

Um framework *web* proporciona meios para resolver todos os pontos mais comuns de uma arquitetura *web*: listagem de dados, internacionalização, formatação, lógica de controle, concorrência, escalabilidade, entre outros.

A RUP e o Scrum podem ser chamados de frameworks de processo. Uma biblioteca poderia ser bem definida como um conjunto de classes ou rotinas que auxiliam em tarefas comuns.

Glossário

dBASE: foi o primeiro Sistema Gerador de Banco de Dados (SGBD) largamente utilizado industrialmente, pelos primeiros microcomputadores baseados em tecnologia Apple Machintosh e IBM PC com sistema operacional DOS.

JVM: Sigla em inglês *Java Virtual Machine*, que em uma tradução livre significa Máquina virtual Java, é um programa que carrega e executa os aplicativos Java, convertendo os bytecodes em código executável de máquina. A JVM é responsável pelo gerenciamento dos aplicativos, à medida que são executados.

Python: Python é uma linguagem de programação orientada a objetos de alto nível, e com grau de abstração relativamente elevado, longe do código de máquina e mais próximo à linguagem humana. Esta linguagem foi criada por Guido Van Rossum em 1991.

Questão para reflexão

Você viu neste módulo sobre a linguagem Javascript e como ela funciona. Para uma melhor compreensão, é importante que você desenvolva uma primeira aplicação utilizando esta linguagem. Sendo assim faça uma pesquisa utilizando a frase "**confirmar cpf em javascript**". Vários *sites* resultarão desta pesquisa.

- Escolha um, verifique o código apresentado, abra seu bloco de notas e implemente o código.
- O objetivo é criar uma página onde o usuário digita seu CPF e o programa em Javascript vai confirmar se este número é valido ou não.
- Salve o arquivo e carregue-o em seu navegador. Nada melhor do que a prática para entender a linguagem, sua sintaxe e seu comportamento.



Considerações Finais

Neste módulo, você aprendeu:

- O que é Javascript.
- Tipo de dados e variáveis.
- Operadores, expressões e comandos em Javascript.
- Documento Object Model.
- Características de bibliotecas e frameworks.

Referências

DEVFURIA. Disponível em: <http://www.devfuria.com.br/javascript/dom/>. Acesso: 10 out. 2016.

FREEDMAN, Alan. **dBASE II para principiantes**. Trad. e revisão técnica Equipe Dialógica; revisão técnica da 2. ed. José Cláudio Boccia. São Paulo: McGraw-Hill, 1987.

MOZILA. Disponível em: <https://developer.mozilla.org/pt-BR/>. Acesso em: 10 out. 2016.

ORACLE. Site Oracle. Disponível em: <http://docs.oracle.com/javase/specs/jls/se8/html/index.html>. Acesso em: 10 out. 2016.

STACKOVERFLOW. Disponível em: <http://pt.stackoverflow.com/questions/17501/qual-%C3%A9-a-diferen%C3%A7a-de-api-biblioteca-e-framework>. Acesso: 10 out. 2016.

W3 – Site do W3 Org. Disponível em: <https://www.w3.org/>. Acesso em: 10 out. 2016.

WIKIVERSITY. Site wikiversity. Disponível em: <https://pt.wikiversity.org/wiki/Introdu%C3%A7%C3%A3o_%C3%A0s_Linguagens_de_Programa%C3%A7%C3%A3o/Python />. Acesso em: 10 out. 2016.

Assista a suas aulas



Aula 5 - Tema: Desvendando o universo *javascript*. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/17ec96c180b69a59be8159a0f9c2c37f>.



Aula 5 - Tema: Desvendando o universo *javascript*. Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/2c27e74738c2e624a5a5afd3faac62d7>.

Questão 1

1. O Javascript é uma linguagem utilizada na criação de páginas para a internet. Assinale a opção correta no que se refere a esta linguagem:

a) Uma linguagem minimalista do próprio Java.

b) É considerada apenas um script de página.

c) É uma linguagem de script orientada a objetos.

d) É uma linguagem compilada.

e) A sintaxe e estrutura do Javascript segue exatamente as regras do Java.

Questão 2

2. O Javascript é uma linguagem utilizada na criação de páginas para a internet.

Assinale a opção que menciona a base desta linguagem:

a) Descende diretamente do próprio Java em regras de sintaxe e estrutura.

b)) É uma linguagem orientada a objetos voltada para o HTML e CSS, usando formas semelhantes de estrutura e sintaxe.

c) Sua sintaxe e estrutura é baseada em linguagens usadas na Web como PHP e MySQL.

d) É uma linguagem baseada em C e C#, chamada de Javascript, pois concorre diretamente com o Java.

e) Descende em sua estrutura de linguagens mais antigas com regras de sintaxe mais dinâmicas e flexíveis como o HyperTalk e Dbase.

Questão 3

3. Assinale a opção que condiz com a forma na qual o Javascript pode ser inserido no código de uma página *web*:

a) Somente de forma interna diretamente no documento HTML.

b) Interno e diretamente no documento HTML, ou chamando o código Javascript através de um arquivo externo.

c) Somente chamando o código Javascript através de um arquivo externo.

d) Criando um app em Javascript e aplicando na página *Web*.

e) Através da *tag* <FORM> do HTML.

Questão 4

4. Existem várias formas de atribuir um valor a uma variável, tanto em Javascript como em todas as linguagens de programação existentes. Assinale a opção que contenha os tipos de declaração de variáveis em Javascript:

a) string, float e int.

b) char, var e const.

c) char, var e int.

d) var boolean e let.

e) var let e const.

Questão 5

5. Assinale a opção que contemple os comandos de controle de fluxo do Javascript:

a) do while, while e for.

b) if, while e for.

c) while, for e goto.

d) else, while e for.

e) while. for e function.

Gabarito

1. Resposta: C.

O JavaScript assemelha-se ao Java, porém não é uma linguagem minimalista do próprio Java. Neste caso, pode-se dizer que ela usa uma sintaxe básica do Java, como convenções de nomenclatura e construções de controle de fluxo, além de ser uma liguagem orientada a objetos

2. Resposta: E.

O JavaScript assemelha-se ao Java, porém não é uma linguagem minimalista do próprio Java. Neste caso, pode-se dizer que ela usa uma sintaxe básica do Java, como convenções de nomenclatura e construções de controle de fluxo, além de ser uma liguagem orientada a objetos.

3. Resposta: B.

O JavaScript pode ser inserido diretamente na página HTML e será interpretado diretamente pelo navegador, ou ser chamado através de um arquivo externo, semelhante ao que acontece com páginas de estilo CSS.

4. Resposta: E.

Existem três tipos de declarações em JavaScript: **var** - Declara uma variável, opcionalmente, inicializando-a com um valor. **let** - Declara uma variável local de escopo do bloco, opcionalmente, inicializando-a com um valor. **const** - Declara uma constante apenas de leitura.

5. Resposta: A.

O Javascript, assim como todas as linguagens de programação, oferece laços condicionais que se iniciam quando uma condição de comparação é colocada, e enquanto esta condição não for atendida, e execução do código vai permanecer processando as informações contidas dentro do laço. O Javascript tem comando de laço semelhantes aos de diversas outras linguagens, sendo os principais do while, while e for.

Unidade 6

Trabalhando com HTML5

Objetivos

1. A linguagem HTML serve de base para a construção de *sites* e páginas para *Web* e integra as diversas tecnologias e linguagens usadas para este fim. Apenas relembrando, em 1990, Tim Berners Lee, que trabalhava no Centro Europeu de Pesquisa Nuclear – **CERN** na Suíça, inventou a linguagem HTML com o objetivo de permitir troca de conteúdos entre diversos centros de pesquisa utilizando a internet. Porém, a internet e suas tecnologias estão em constante evolução, e a linguagem HTML não poderia ficar de fora desse processo. Neste módulo, você verá como o HTML5 melhorou o funcionamento dos *sites* através de novos elementos e definições de sua linguagem, ou seja, Anatomia e Formulários, Áudio e Vídeo, Canvas, SVG.

Introdução

O W3C, como já visto, é a instituição internacional que orienta a internet, suas tecnologias, padrões de *hardware*, linguagens utilizadas para a construção de *sites* e páginas. Estas recomendações, em sua maioria, são adotadas como padrão pelos diversos fabricantes e empresas de tecnologia, bem como tornam-se padrões das diversas linguagens de desenvolvimento usadas na construção de *sites* e páginas *Web*. O W3C conta com a colaboração de instituições públicas e privadas de todas as partes do mundo.

Dessa forma, foi fundado em 2004, o "*WHATWG – Web Hypertext Application Technology Working Group*", um grupo criado por desenvolvedores das principais empresas da internet, como Mozila, Opera, Apple, entre outros. Nesta ocasião, o W3C partia para o desenvolvimento de uma segunda versão para a linguagem XHTML, enquanto que os membros da "WHATWG" trabalhavam em uma nova versão da linguagem HTML com mais flexibilidade para a produção de *sites* e sistemas para a *Web*, e que foi chamada de HTML5.

Em 2006, o W3C reconheceu o trabalho desenvolvido pelo WHATWG e Tim Berners Lee juntou-se ao grupo para **o desenvolvimento do HTML5**, deixando assim o desenvolvimento do XHTML2. O grupo que estava desenvolvendo o XHTML2 foi descontinuado em 2009, tornando o HTML5 a linguagem que seria continuada e é aplicada até os dias atuais (W3C, s.d.).

Link

Conheça em detalhes a história do surgimento do HTML5 contada pelos próprios criadores em: <https://www.w3.org/TR/html5/introduction.html#history-0>. Acesso em: 10 out. 2016.

1. Anatomia e formulários

1.1 anatomia

O HTML5 teve suas regras de marcação definidas pelo WHATWG, bem como as regras que definem as APIs que formam a base da arquitetura dos *sites* e páginas *Web*.

API, que vem da sigla em inglês de *"Application Programming Interface"*, ou numa tradução livre "Interface de Programação de Aplicativos", são padrões de programação que permitem a construção de aplicativos. Por exemplo, uma API poderá interligar diversas funções de um *site*, imagens, textos, vídeos e mapas para que estes possam ser utilizados em outras aplicações dentro da *web* (TECMUNDO, s.d.).

Através de suas APIs, o HTML5 melhorou as descrições para o uso do CSS e do Javascript para a formatação visual dos *sites*, tornando um *site* ou aplicação *web* mais leve e funcional. Além disto o HTML5 criou novas *tags* e adequou a função de outras, por exemplo, nas versões mais antigas do HTML não existia um padrão para criação de seções comuns como menus, barras de navegação, cabeçalhos e rodapés (MOZILA, s.d.).

O HTML5 apresentou uma nova maneira de escrever seu código e organizar uma página *web*, atentando aos conceitos de semântica, interatividade e interoperabilidade com menos instalações de *plugins*, ganhando em rendimento e performance dos dispositivos usados e se preparando também para o uso dos novos dispositivos e possibilidades que constantemente surgem na *Web*. Além disto tudo, o HTML5 tem em sua definição que deve manter a compatibilidade com as versões anteriores do HTML, ou seja, os navegadores ao ler a definição do "<Doctype HTML>" aceitam a nova versão HTML5 bem como as suas versões anteriores (W3C, s.d.).

1.2 Formulários

Um formulário é basicamente o local onde o usuário entra com determinada informação dentro de um *site* através de campos de texto, botões e caixas. O usuário interage com estes formulários e as informações normalmente são processadas e enviadas para o servidor gerando diversos tipos de ações, como retorno de buscas, o resultado de cálculos, saldos bancários entre vários outros tipos. Existem também formulários que não trabalham com o envio de dados para um servidor, executando ações diretamente no lado cliente, como o caso de várias funções em Javascript que se utiizam de entrada de dados.

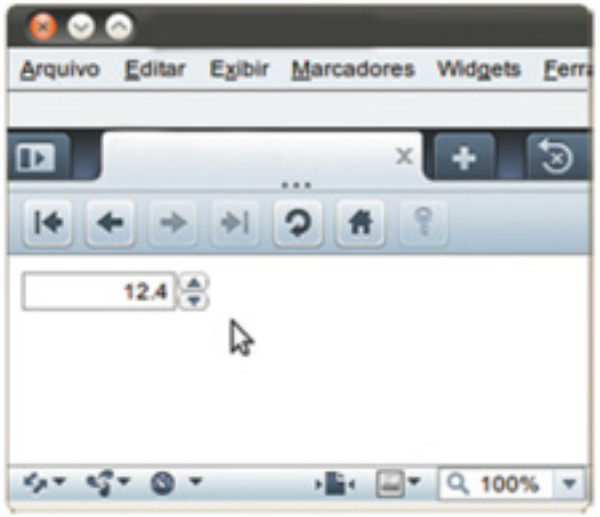
Os formulários em HTML5 foram reestru-

turados e ganharam novos recursos através do elemento "*input*". O elemento *input* aceita em seu atributo "type", valores como "tel" para formatação de números de telefone, "search" – um campo de busca, "*e-mail*" com formatação e validação. Você poderá notar em alguns exemplos de códigos que geram entradas e campos com muito mais flexibilidade do que em versões anteriores do HTML (W3C, s.d.).

Veja o exemplo de código:

```
<form>
<imput name="valuex" type="number"
value="12.4" step="0.2"
min="0" Max="20" />
</form>
```

Este código gera um Box Numérico, mostrando o número 12.4, que vai de 0 a 20 incrementado de 0,2 em 0,2 em sua escala, desta forma:



É importante ressaltar que todos os elementos de um formulário, mesmo os novos, devem ser colocados dentro da *tag* <form> </form>.

Para saber mais

Um formulário é um componente de uma página da *Web* que tem vários tipos de campos, tais como campos de texto, botões, caixas e controles de intervalo. Um usuário pode interagir com formulários e fornecer dados que podem ser enviados para o servidor para processamento adicional (por exemplo, retornando os resultados de uma busca ou de um cálculo).



Link

Conheça mais sobre Formulários HTML5: <https://www.w3.org/TR/html5/forms.html#introduction-1>. Acesso em: 10 out. 2016.

2. Áudio e vídeo

2.1 Áudio

Vários recursos encontravam dificuldades de serem implementados em páginas HTML, como o caso de áudio e vídeos. No HTML5 foram introduzidos elementos que controlam estes componentes de forma fácil e direta. Veja o caso do código que insere um áudio:

```
<audio src="musica.mp3"
controls="true" autoplay="true" />
```

Neste simples código, o valor de "controls" instrui que o áudio será iniciado, com controle de áudio através de botões de "play", "pause", "volume", "barra de progresso", "contagem de tempo" e assim por diante.

2.2 Vídeo

Assim como em áudio, o elemento de vídeo conta com um componente para exibição. Você pode notar que a sua aplicação é bastante simples, e cabe ressaltar que aqui encontramos as definições do tamanho da caixa de vídeo que será apresentado, definido pelos valores em pixels de “width” e “height”.

```
<video src=”meuvídeo.wmv”
width=”400” height=”300” />
```

Para saber mais

O elemento vídeo do HTML5 faz o *download* de todo o arquivo do vídeo e inicia sua reprodução logo após terminado o *download*. Existe outra maneira que é bastante usada na *Web* atualmente que é a apresentação de vídeo em tempo real de transmissão, chamada de “*Streaming*”.

Link

Você poderá encontrar mais explicações e exemplos sobre áudio e vídeo em HTML5: <https://www.w3.org/TR/html5/embedded-content-0.html#audiotracklist-and-videotracklist-objects>. Acesso em: 10 out. 2016.

3. Canvas

Canvas é um elemento HTML5, uma API que permite desenhar na tela do computador utilizando os recursos da linguagem Javascript. Canvas, que em inglês significa tela, gera um espaço onde pode ser inserido um elemento gerado pela linguagem Javascript, sendo possível desenhar texto, sombras, gradientes, incluir imagens, transformar objetos e vários outros recursos (MOZILA, s.d.).

4. SVG

SVG é uma linguagem de marcação de gráficos vetoriais em XML e que é aceita pela recomendação do HTML5. Os gráficos SVG podem se adaptar a diferentes resoluções de tela e reutilizados em diferentes tamanhos em diversas páginas *web*. Outra vantagem é que se pode criar gráficos vetoriais utilizando os conhecidos **programas vetoriais** e exportando diretamente os gráficos em padrão SVG para que possam ser aplicados em páginas *web* (W3C, s.d.).

Para saber mais

Existem vários programas que trabalham com imagens vetoriais, como os programas da linha CAD, AutoCAD, Studio 3DMax, programas de ilustração como o Illustrator e o CorelDraw, dentre muitos outros que podem exportar seus arquivos diretamente no padrão SVG.

Link

Você encontra a descrição completa sobre SVG em: <https://www.w3.org/TR/SVG11/concepts.html>. Acesso em: 10 out. 2016.

Glossário

CERN: A Organização Europeia para a Pesquisa Nuclear (em francês: *Organisation Européenne pour la Recherche Nucléaire*), conhecida como CERN (antigo acrônimo para *Conseil Européen pour la Recherche Nucléaire*) é o maior laboratório de física de partículas do mundo, localizado em Meyrin, na região em Genebra, na fronteira Franco-Suíça. (CERN).

Streaming: Quando o usuário executar um *streaming* de vídeo, o player de vídeo vai começar o processo de transferência de parte do vídeo, será baixada uma pequena parte e o vídeo será iniciado, o servidor continuará a enviar todo o resto do arquivo de vídeo enquanto o usuário assiste, assim ele não tem que esperar até que o vídeo termine a transferência para que possa começar a assistir. (SITEHOSTING).

SVG: Abreviatura do termo em inglês Scalable Vector Graphics.

Questão para reflexão

Você viu neste módulo novos recursos do HTML5 no envio de formulário e controle de áudio e vídeo. Agora é a oportunidade de testar e ver como isto funciona. Grave uma mensagem em vídeo usando seu computador ou até mesmo seu *smartphone*. Com o vídeo gravado, monte uma página que possa carregar este vídeo, com os comandos descritos nesta aula. Aproveite e na mesma página elabore um formulário de participação, onde seu usuário possa informar seu nome e endereço de *e-mail*, e ao clicar em um botão "enviar" seu formulário possa submeter o conteúdo para um *e-mail* previamente definido para esta finalidade, ou seja, receber as informações deste formulário.



Considerações Finais

Neste módulo, você aprendeu:

- Anatomia do HTML5 e novos recursos de formulários.
- Áudio e vídeo em HTML5.
- Canvas.
- SVG.

Referências

CERN. Site CERN Disponível em: <https://home.cern/about>. Acesso em: 10 out. 2016.

MOZILA. Disponível em: <https://developer.mozilla.org/pt-BR/>. Acesso em: 10 out. 2016.

SITEHOSTING. Disponível em: <http://www.sitehosting.com.br/como-funciona-o-streaming/>. Acesso em: 10 out. 2016.

TECMUNDO. Site Tecmundo. Disponível em: <http://www.tecmundo.com.br/programacao/1807-o-que-e-api-.htm>. Acesso em: 10 out. 2016.

W3. Site do W3 Org. Disponível em: <https://www.w3.org/>. Acesso em: 10 out. 2016.

Assista a suas aulas



Aula 6 - Tema: Trabalhando com HTML5. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f-1d/1c86120ccfcd76e6c087a34e698fd25c>.



Aula 6 - Tema: Trabalhando com HTML5. Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/e80d34967dc620aa718f8665d0d7a52e>.

Questão 1

1. Assinale a alternativa correta. O HTML5 permite a utilização de APIs, que na prática são:

a) Os aplicativos criados em outras linguagens e que são inseridos na página.

b) Os aplicativos distribuídos na internet, por exemplo, pelo Google Play e que são inseridos na página.

c) Os aplicativos criados exclusivamente com HTML e Javascript.

d) Os padrões de programação que permitem a construção de aplicativos sem que isto seja percebido pelo usuário.

e) As janelas que podem ser inseridas no *site* e que se pode inserir por exemplo, um vídeo ou um mapa no *site*.

Questão 2

2. Assinale a alternativa correta. O HTML5 é a versão mais atual da linguagem HTML. Pode-se dizer que esta versão é:

a) Uma evolução do XHTML.

b) Uma nova versão da linguagem HTML com mais flexibilidade para a produção de *sites* e sistemas para a *Web*.

c) Uma junção de XHTML e Javascript.

d) Uma junção de HTML, CSS e Javascript.

e) Uma nova versão da linguagem HTML com regras mais rígidas e de orientação a objetos para a produção de *sites* e sistemas para a *Web*.

Questão 3

3. Assinale a alternativa correta. Os formulários foram reestruturados em HTML5 e ganharam novos recursos através do elemento:

a) input password.

b) text area.

c) input.

d) e-mail field.

e) string.

Questão 4

4. Assinale a alternativa que corresponde à função do elemento Canvas em HTML5:

a) Permitir desenhar na tela do computador utilizando recursos da linguagem Javascript.

b) Permitir desenhar na tela do computador utilizando recursos da linguagem HTML.

c) Permitir desenhar na tela do computador utilizando recursos de programas editores de imagens Bitmap.

d) Permitir importar desenhos na tela do computador utilizando o HTML.

e) Permitir desenhar na tela do computador utilizando recursos de programas vetoriais 3D.

Questão 5

5. Assinale a opção que descreva o que é SVG:

a) Uma linguagem de marcação de gráficos vetoriais em Javascript e que é aceita pela recomendação do HTML5.

b) Uma linguagem de marcação de gráficos vetoriais originada pelo Autocad e que é aceita pela recomendação do HTML5.

c) Um elemento HTML que permite desenhar na tela do computador utilizando recursos de programas vetoriais.

d) Uma linguagem de marcação de gráficos vetoriais em XML e que é aceita pela recomendação do HTML5.

e) Um elemento HTML que permite desenhar na tela do computador utilizando recursos de programas vetoriais especificamente em 3D.

Gabarito

1. Resposta: D.

API, que vem da sigla em inglês de "*Application Programming Interface*", ou em uma tradução livre "Interface de Programação de Aplicativos" são padrões de programação que permitem a construção de aplicativos e que isto não seja percebido pelo usuário.

2. Resposta: B.

O "*WHATWG – Web Hypertext Application Technology Working Group*", um grupo criado por desenvolvedores das principais empresas da internet, como Mozila, Opera, Apple, entre outros, no ano de 2004 trabalhavam em uma nova versão da linguagem HTML com mais flexibilidade para a produção de *sites* e sistemas para a *Web*, e que foi chamada de HTML5. Em 2006 o W3C reconheceu o trabalho desenvolvido pelo WHATWG.

3. Resposta: C.

Os formulários em HTML5 foram reestruturados e ganharam novos recursos através do elemento "*input*". O elemento *input* aceita em seu atributo "type", valores como "tel" para formatação de números de telefone, "search" – um campo de busca, "*e-mail*" com formatação e validação e muitos outros.

4. Resposta: A.

Canvas é um elemento HTML5, uma API que permite desenhar na tela do computador utilizando os recursos da linguagem Javascript. Canvas, que em inglês significa tela, gera um espaço onde pode ser inserido um elemento gerado pela linguagem Javascript, sendo possível desenhar texto, sombras, gradientes, incluir imagens, transformar objetos e vários outros recursos.

5. Resposta: D.

SVG, uma abreviatura do termo em inglês Scalable Vector Graphics é uma linguagem de marcação de gráficos vetoriais em XML e que é aceita pela recomendação do HTML5.

Os gráficos SVG podem se adaptar às diferentes resoluções de tela e reutilizado em diferentes tamanhos em diversas páginas *web*.

Unidade 7

Recursos avançados de HTML5

Objetivos

1. Existem recursos que foram aprimorados ou implementados com o HTML5, novas possibilidades de carregamento e visualização de conteúdos, sua visualização estando ***off-line*** e muitos outros. Você verá, neste módulo, aspectos relacionados com aplicações *off-line* e *Web Workers*, geolocalização, Server Sent Events e, ainda, Websocket e Cross Document Messaging.

Introdução

O HTML5 proporciona novas possibilidades de carregamento de páginas pelos navegadores, como carregamentos de páginas estando o usuário *off-line*, e um controle maior de atualizações de conteúdo dos *sites*. Saber o que cada recurso proporciona torna você um profissional melhor preparado para as novas demandas oferecidas ao desenvolvedor de *sites* e aplicações *Web*.

1. Aplicações *off-line* e *web workers*

1.1. Aplicações *off-line*

Desde o início da internet estabeleceu-se uma relação direta entre páginas e *sites* que deveriam ser hospedadas em *servidores* e baixadas (feito o *download*) pelos navegadores, e para isto o computador que estivesse acessando a página deveria estar conectado à *Web*. O HTML5 possibilita que páginas sejam armazenadas nos dispositivos que as acessam em uma área de **cache**, que acumula arquivos temporários nos dispositivos de memória de armazenamento dos equipamentos. A partir disto, estas páginas podem ser acessadas com suas características originais mantidas mesmo estando o usuário desconectado da internet (MOZILA, s.d.).

Para que isto possa funcionar é necessário escrever um arquivo de manifesto, indicando quais URLs (páginas do *site*) que devem ser colocadas em cache. O navegador irá fazer o *download* destas páginas e mantê-las atualizadas no cachê de seu dispositivo, e quando você tentar acessá-las estando *of-*

f-line, o navegador irá automaticamente apontar para as cópias locais (MOZILA, s.d.).

Veja o exemplo de código, a seguir:

```
Arquivo calendario.html
<!DOCTYPE HTML>
<html>
 <head>
    <title>Calendario</title>
    <script src="calendario.js"></
script>
     <link rel="stylesheet"
href="calendario.css">
 </head>
 <body>
     <p>a data é: <output
id="calendario"></output></p>
 </body>
</html>
```

Esta página carrega um *widget* de um calendário e para isto depende do carregamento dos arquivos "calendário.js" e "calendário.css". Para que esta página possa ser carregada *off-line* deve-se criar o manifesto, que é um arquivo com a extensão ".manifest" e que contém a lista de arquivos que devem ser mantidos no cache para que sejam carregados *off-line*. No nosso caso, este arquivo será escrito dessa forma:

```
CACHE MANIFEST
calendário.html
calendário.css
calendário.js
```

O novo código para carregamento *off-line*

desta página ficará assim:

```
Arquivo calendario.html com a
chamada de monifesto
<!DOCTYPE HTML>
<html manifest="calendario.
manifest">
    <head>
        <title>calendario</title>
        <script src="calendario.
js"></script>
        <link rel="stylesheet"
href="calendario.css">
    </head>
    <body>
        <p>a data é : <output
id="calendario"></output></p>
    </body>
</html>
```

Para saber mais

Baseando-se no exemplo apresentado, faça um novo código carregando um novo "*widget*", um relógio por exemplo. Aproveite e verifique através de sua busca na internet a vasta quantidade de componentes que podem ser inseridos em um *site* e que podem ser carregados *off-line*.

1.2 *Web workers*

Web worker ou workers é um recurso do HTML5 que nos permite executar javascript em segundo plano, independente de qual interface ou aplicativo seja rodado naquele momento. Este recurso é bastante útil, pois permite que *scripts* que não dependam de

interação com o usuário possam ser carregados e executados enquanto as aplicações destinadas aos usuários rodam, evitando assim interrupções de funcionamento dentro das aplicações *web* (MOZILA, s.d.).

Os ***Web workers*** são relativamente pesados e sua execução normalmente é demorada. Um bom exemplo de *Web workers* acontece quando o usuário está usando a *Web* e é solicitada a atualização de algum aplicativo. Esta atualização acontece em segundo plano e proporciona ao usuário a possibilidade de continuar navegando ao mesmo tempo que estas atualizações são carregadas (W3C, s.d.).

Para saber mais

Os *Web workers* são muito comuns e passam quase que despercebidos pelos usuários. Por exemplo, quando você usa um computador ou mobile e o seu antivírus começa uma atualização, neste momento, a atualização é um recurso Web worker sendo aplicado.

Link

Conheça mais sobre *Web workers* HTML5 em: <https://html.spec.whatwg.org/multipage/workers.html#workers>. Acesso em: 10 out. 2016.

2. Geolocalização

O HTML5 está preparado a oferecer recursos de geoposicionamento através de uma API. Existem três principais tipos de geoposicionamento:

- Geoposicionamento por IP – Usados pela maioria dos navegadores, determina a cidade ou região que você se encontra.
- Triangulação GPRS – verifica o posicionamento de dispositivos conectados a uma rede de celulares, determinando sua posição triangulando as posições das antenas de GPRS. Mais preciso, determina qual é o bairro em que você se encontra.
- **GPS** – é o que tem maior precisão, com uma margem de erro de cinco metros estando em condições ideais (MOZILA, s.d.).

O HTML5 mantém o Geolocation API, que simplifica o uso de geoposicionamento em *sites*. Para usar este API você deve obter a posição do usuário, basta aplicar o script:

```
navigator.geolocation.
getCurrentPosition(showpos)
```

Você pode observar neste código o parâmetro *showpos*, que é na verdade uma função de *callback*, ou seja, vai receber um objeto de geoposicionamento, o que na prática faz retornar à posição correta do usuário dada em coordenadas de latitude e longitude. Isto pode ser notado através do código da função *showpos*, como segue:

```
function showpos(position){
lat=position.coords.latitude
lon=position.coords.longitude
```

Verifique o código a seguir, copie-o e cole-o no bloco de notas com o nome localização.html.

```
<html lang="en">
<head>
    <title>Geolocation demo</title>
    <meta charset="utf-8" />
</head>
<body>
    <h1>Geolocation demo</h1>
    <p>
        Find out approximately where you are.
    </p>
    <p>
```

```
        Step 1: <button onclick="GetMap()">Show map</button>
    </p>
    <p>
        Step 2: When prompted, allow your location to be shared to see
Geolocation in action
    </p>
    <div id="mapDiv" style="position: relative; width: 800px; height:
600px;"></div>
    <script type="text/javascript" src="http://ecn.dev.virtualearth.net/
mapcontrol/mapcontrol.ashx?v=7.0"></script>
    <script type="text/javascript">
        var map = null;
        function GetMap() {
            /* Replace YOUR_BING_MAPS_KEY with your own credentials.
                Obtain a key by signing up for a developer account at
                http://www.microsoft.com/maps/developers/ */
            var cred = "YOUR_BING_MAPS_KEY";
```

```
            // Initialize map
            map = new Microsoft.Maps.Map(document.getElementById("mapDiv"),
                { credentials: cred });
            // Check if browser supports geolocation
            if (navigator.geolocation) {
                navigator.geolocation.getCurrentPosition(locateSuccess,
locateFail);
            }
            else {
                alert('I\'m sorry, but Geolocation is not supported in
your current browser. Have you tried running this demo in IE9?');
            }
        }
        // Successful geolocation
        function locateSuccess(loc) {
            // Set the user's location
            var userLocation = new Microsoft.Maps.Location(loc.coords.
```

```
latitude, loc.coords.longitude);
            // Zoom in on user's location on map
            map.setView({ center: userLocation, zoom: 17 });
            // Draw circle of area where user is located
            var locationArea = drawCircle(userLocation);
            map.entities.push(locationArea);
        }
        // Unsuccessful geolocation
        function locateFail(geoPositionError) {
            switch (geoPositionError.code) {
                case 0: // UNKNOWN_ERROR
                    alert('An unknown error occurred, sorry');
                    break;
                case 1: // PERMISSION_DENIED
                    alert('Permission to use Geolocation was denied');
                    break;
                case 2: // POSITION_UNAVAILABLE
```

```
                    alert('Couldn\'t find you...');
                    break;
                case 3: // TIMEOUT
                    alert('The Geolocation request took too long and timed
out');
                    break;
                default:
            }
        }
        // Draw blue circle on top of user's location
        function drawCircle(loc) {
            var radius = 100;
            var R = 6378137;
            var lat = (loc.latitude * Math.PI) / 180;
            var lon = (loc.longitude * Math.PI) / 180;
            var d = parseFloat(radius) / R;
            var locs = new Array();
```

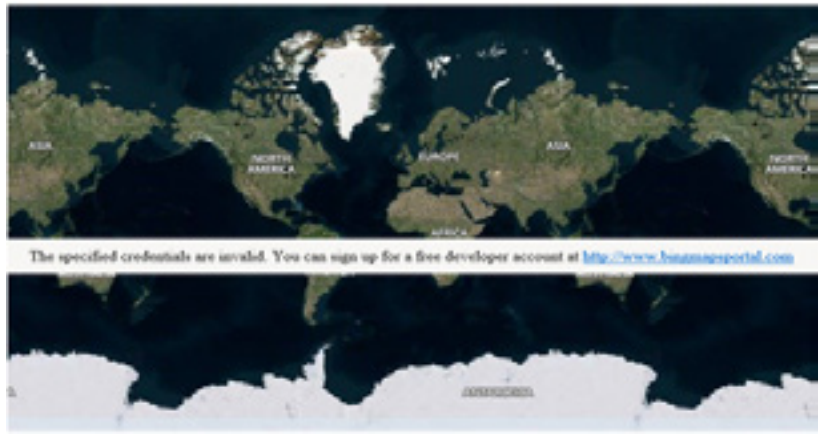
```
            for (x = 0; x <= 360; x++) {
                var p = new Microsoft.Maps.Location();
                brng = x * Math.PI / 180;
                p.latitude = Math.asin(Math.sin(lat) * Math.cos(d) + Math.
cos(lat) * Math.sin(d) * Math.cos(brng));
                p.longitude = ((lon + Math.atan2(Math.sin(brng) * Math.
sin(d) * Math.cos(lat), Math.cos(d) - Math.sin(lat) * Math.sin(p.
latitude))) * 180) / Math.PI;
                p.latitude = (p.latitude * 180) / Math.PI;
                locs.push(p);
            }
            return new Microsoft.Maps.Polygon(locs, { fillColor: new
Microsoft.Maps.Color(125, 0, 0, 255), strokeColor: new Microsoft.Maps.
Color(0, 0, 0, 255) });
        }
    </script>
<body>
<html>
```

Em seguida, execute o código (abrir com internet explorer), permita o conteúdo bloqueado. A seguinte imagem será apresentada:



O resultado de execução do código na imagem:

Figura 3 - Imagem resultante da linha de códiogo da geolocalização



Fonte: http://javeirosonline.blogspot.com.br/2014/06/html-5-web-workers.html

Podendo ser ampliado para encontrar a sua localização atual.

3. Server sent events

Os Servers Sent Events, termo em inglês, que em uma tradução livre significa Eventos Enviados pelo Servidor, são atualizações que uma página *Web* recebe automaticamente de um servidor.

Estes eventos de atualização já aconteciam em versões HTML anteriores ao HTML5, mas a página tinha que executar uma chamada ao servidor e verificar se havia atualizações disponíveis. Este processo tornou-se muito mais dinâmico através do HTML5, pois com os eventos enviados pelo servidor, estas atualizações são mandadas automaticamente.

Podemos observar estas atualizações em portais de notícias, *sites* de conteúdo, *feed* de notícias e até mesmo em redes sociais, como o *Facebook* e o *Twitter*, por exemplo. É importante ressaltar que esta atualização depende também de os navegadores serem compatíveis com este tipo de atualização automática (W3C, s.d.).

Para saber mais

Você deve se atentar a quais navegadores não suportam este recurso, pois este é um detalhe que pode comprometer seu projeto final ou as intenções de alcance de audiência para o conteúdo apresentado no *site*. Disponível em: <http://www.w3schools.com/html/html5_serversentevents.asp>. Acesso em: 10 out. 2016.

4. WebSocket e cross document messaging

4.1 WebSocket

Os WebSockets são uma nova especificação do HTML5 que permite um servidor estabelecer uma conexão diretamente com o navegador, comunicando-se diretamente com ele e permitindo que haja uma troca direta de dados entre o servidor e o cliente, que, neste caso, é o próprio navegador *web* (W3C, s.d.).

Esta conexão era feita pelas versões anteriores do HTML seguindo as recomendações de funcionamento da própria *Web*, que estabeleciam a necessidade de uma série de solicitações e respostas entre o navegador e o servidor em aplicações que necessitassem de atualizações em tempo real. Com a utilização dos WebSockets, a conexão entre navegador e servidor é estabelecida uma única vez, evitando problemas de atrasos nesta comunicação devido a solicitações e suas respectivas respostas. Alguns exemplos de conexões diretas entre servidores e navegadores são encontrados em aplicações de jogos *on-line, chats* e muitos outros.

Quando os **WebSockets** são utilizados, o cliente (navegador) estabelece uma conexão através de um processo de envio de pedido via HTTP para o servidor, onde um cabeçalho de atualização é inserido neste pedido e informa ao servidor que o cliente deseja estabelecer uma conexão WebSocket (MOZILA, s.d.).

Este cabeçalho pode ser escrito desta forma:

```
GET ws://exemplo.websocket.com.br/
HTTP/1.1
  Origin: http://exemplo.com
  Connection: Upgrade
  Host: exemplo.websocket.com
  Upgrade: websockets
```

4.2 Cross document messaging

Este é um recurso oferecido pelo HTML5 que permite uma comunicação cruzada entre páginas *Web* mantendo sua segurança, utilizando o método window.postMessage. Normalmente, os *scripts* em páginas diferentes estão autorizados a acessar um ao outro se e somente se as páginas executadas estiverem em locais com o mesmo protocolo (normalmente dois https), número da porta (443 sendo o padrão para https) e *host*, sendo definido para ambas as páginas o mesmo valor (W3C, s.d.).

O método window.postMessage fornece um mecanismo controlado de contornar essa restrição de uma forma segura quando usado corretamente. Quando chamado, este método cria um MessageEvent para ser despachado para a página de destino e proporcionar esta comunicação atendendo a regras de segurança previamente estabelecidas para seu uso (MOZILA, s.d.).

Link

Você encontra uma descrição mais aprofundada sobre Cross Document Messaging em: <https://developer.mozilla.org/en-US/docs/Web/API/Window/postMessage>. Acesso em: 10 out. 2016.

Glossário

Off-line: termo da língua inglesa cujo significado literal é "fora de linha", ou em uma tradução livre "desconectado". É habitualmente usado para designar que um determinado usuário da internet ou de uma outra rede de computadores não está conectado à rede.

Cache: área de memória onde é mantida uma cópia temporária de dados armazenados em um meio de acesso mais lento, com o objetivo de acelerar a recuperação dos dados.

GPS: é a sigla para *Global Positioning System*, que em português significa "Sistema de Posicionamento Global", e consiste numa tecnologia de localização por satélite.

Questão para reflexão



Você viu neste módulo os principais recursos avançados da linguagem HTML5.

Desenvolva uma pesquisa entre os *sites* da internet e identifique pelos menos três *sites* ou portais para cada item visto que se utilizam destes recursos. Isto ajudará você a desenvolver a percepção destes recursos, pois eles normalmente são executados entre os navegadores e os servidores.

Considerações Finais

Neste módulo, você aprendeu:

- Aplicações *Off-line* e *Web Workers*.
- Geolocalização.
- Server Sent Events.
- Websocket e Cross Document Messaging.

Referências

DEVMEDIA. Aplicações de websockets. Disponível em: <http://www.devmedia.com.br/uso-de--websockets-e-html5/32267>. Acesso: out. 2016.

HTML5. Adendos: web workers. Disponível em: <https://www.html5rocks.com/pt/tutorials/workers/basics/>. Acesso em: 10 out. 2016.

MOZILA. Disponível em: <https://developer.mozilla.org/pt-BR/>. Acesso em: 10 out. 2016.

SIGNIFICADOS. Disponível em: <http://www.significados.com.br/>. Acesso em: 10 out. 2016.

TECMUNDO. Disponível em: <http://www.tecmundo.com.br/navegador/201-o-que-e-cache-.htm>. Acesso em: 10 out. 2016.

W3. Disponível em: <https://www.w3.org/>. Acesso em: 10 out. 2016.

Assista a suas aulas



Aula 7 - Tema: Recursos avançados de HTML5. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/680ffa1c04cd464efc17ef367c3d479f>.



Aula 7 - Tema: Recursos avançados de HTML5. Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f-1d/10c3c4722e56c5eae6ca3691bb47119b>.

Questão 1

1. O HTML5 permite que páginas da internet sejam acessadas quando o computador estiver desconectado da rede, ou seja, quando estiver *"off-line"*.

Assinale a opção correta para tornar isto uma possibilidade.

a) Deve-se criar o manifesto, que é um arquivo com a extensão ".manifest" e chamá-lo dentro do código HTML.

b) Deve-se criar o manifesto, que é um código em XML dentro do código HTML.

c) Deve-se criar o manifesto, que é um arquivo em XHTML externo.

d) Deve-se criar o manifesto, que é um código em CSS dentro do código HTML,

e) Deve-se criar o manifesto, que é um arquivo em Javascript externo.

Questão 2

2. Assinale a opção que indica o que são na prática os Web workers, ou em uma tradução livre os trabalhadores *web*.

a) APIs que proporcionam a execução de *scripts* em segundo plano e que interrompe a execução normal das demais aplicações.

b) São *scripts* em HTML que proporcionam a execução de aplicativos externos, como vídeos e outros.

c) APIs que proporcionam a execução de *scripts* em segundo plano e que mantém a execução normal das demais aplicações.

d) São *scripts* em Javascript que proporcionam a execução de aplicativos externos, como vídeos e outros.

e) São os interpretadores dos navegadores que fazem a leitura dos códigos nas diversas linguagens, como Javascript, CSS, PHP e outros.

Questão 3

3. O HTML5 está preparado a oferecer recursos de geoposicionamento através de uma API. Assinale a opção correta que diz respeito ao que são os TRE principais sistemas de geoposicionamento.

a) Geoposicionamento por local físico, Triangulação GPRS e GPS.

b) Geoposicionamento por IP, Triangulação de Satélites e GPS.

c) Geoposicionamento de Servidores, Triangulação de redes WI-FI e GPS.

d) Geoposicionamento por IP, Triangulação GPRS e GPS.

e) Geoposicionamento por IP, Triangulação de Servidores e GPS.

Questão 4

4. Assinale a opção que contempla o que são Servers Sent Events.

a) *Scripts* de uma página *Web* que são processados pelo lado do servidor.

b) Atualizações que uma página *Web* recebe automaticamente de um servidor.

c) Scripts da linguagem Javascript que são processados pelo um servidor.

d) Atualizações que uma página *Web* solicita a cada tempo determinado a um servidor.

e) Atualizações que uma página *Web* recebe de um servidor para apresentação de vídeos.

Questão 5

5. Os WebSockets são uma nova especificação do HTML5 que permitem um servidor:

a) Enviar o resultado de dados processados no servidor para que o navegador possa fazer o *download*.

b) Identificar endereços de IP da máquina que está visualizando o *site*.

c) Processar os *scripts* da linguagem Javascript do *site* acessado.

d) Atualizar uma página *Web* acessada por um navegador.

e) Estabelecer uma conexão diretamente com o navegador, comunicando-se diretamente com ele e permitindo que haja uma troca direta de dados entre o servidor e o cliente.

Gabarito

1. Resposta: A.

O HTM5 possibilita que páginas sejam armazenadas nos dispositivos que as acessa em uma área de cache, que acumula arquivos temporários nos dispositivos de memória de armazenamento dos equipamentos. Para que isto possa funcionar é necessário escrever um arquivo de manifesto, indicando quais URLs (páginas do *site*) que devem ser colocadas em cache.

2. Resposta: C.

Os Web Workers, ou em uma tradução livre os trabalhadores *web*, são APIs que proporcionam a execução de *scripts* em segundo plano, independente de qual interface ou aplicativo esteja sendo rodado naquele momento, não interferindo na sua execução.

3. Resposta: D.

O HTML5 está preparado a oferecer recursos de geoposicionamento através de uma API. 4. Existem três sistemas principais de geoposicionamento: Geoposicionamento por IP – usados pela maioria dos navegadores, determina a cidade ou região em que você se encontra. Triangulação GPRS – verifica o posicionamento de dispositivos conectados a uma rede de celulares, determinando sua posição triangulando as posições das antenas de GPRS. GPS – é o que tem maior precisão, com uma margem de erro de cinco metros estando em condições ideais.

4. Resposta: B.

Servers Sent Events, termo em inglês que em uma tradução livre significa “Eventos Enviados pelo Servidor”, são atualizações que uma página *Web* recebe automaticamente de um servidor.

5. Resposta: E.

Os WebSockets são uma nova especificação do HTML5 que permitem a um servidor estabelecer uma conexão diretamente com o navegador, comunicando-se diretamente com ele e permitindo que haja uma troca direta de dados entre o servidor e o cliente, que neste caso é o próprio navegador *web*.

Unidade 8

Trabalhando com metadata e semântica

Objetivos

1. Em HTML, os metadados sempre estiveram presentes como um conjunto de informações sobre a página *Web* e seu conteúdo. Porém, com a chegada do HTML5, o termo “metadados” ou “***Meta Tags***” tornou-se mais amplo. Você verá neste módulo: O que são metadados em HTML5, metadados de navegação, metadados da página, semântica das páginas.

Introdução

A chegada do HTML5 fez com que o termo "metadados" se tornasse mais amplo. Isto acontece levando-se em conta a ação de "linkar" as páginas da internet, recurso este que sempre esteve presente e até define a própria invenção do HTML e do funcionamento da internet.

1. O que são metadados em HTML5

Desde o início da internet estabeleceu-se uma relação direta entre páginas através de *links*. Para gerar um *link* entre páginas inicialmente existiam dois tipos de elementos, a *tag* <a> ou o elemento *link*, demonstrados nos códigos a seguir:

```
<a href="http://www.agenciatur.
com.br">Visite a página de viagens
</a>
<link rel="stylesheet"
href="estilo.css" />
```

O elemento *link*, neste caso, cria um **metadado**, um *link* que não é mostrado no conteúdo, mas o usuário o usa de outras formas. O uso mais comum para este elemento é vincular um documento a uma folha de estilos (W3C, s.d.).

Os elementos da categoria Metadata devem ser colocados na seção <head> do código HTML e são:

- base
- command

- link
- meta
- noscript
- script
- style
- title

Para saber mais

Metadados trazem informações sobre a página e seu conteúdo. Estas informações são utilizadas pelos navegadores e *sites* de busca classificando e "rankeando" seu *site* de acordo com estes dados.

Link

Conheça mais sobre metadados em HTML5. Disponível em: <https://www.w3.org/TR/html5/document-metadata.html>. Acesso em: 10 out. 2016.

2. Metadados de navegação

O elemento de navegação é inserido em HTML5 através da tag <nav>, que marca a seção na página que aponta para outras páginas, basicamente a seção onde é descrita a barra de navegação do site e que será repetida nas demais páginas.

Porém, como já foi mencionado, estes *links* criam metadados que não são mostrados no conteúdo, mas servem ao usuário. Além disto, existem vários parâmetros para o uso destes metadados, sendo os de navegação:

- archives
- author
- bookmark
- first
- help
- index
- last
- license
- next
- prefetch
- prev
- search
- up

3. Metadados da página

Os metadados são informações adicionais acerca dos conteúdos. Esta informação resume, enriquece ou complementa os conteúdos, produzindo assim um potencial incremento de informação. Os metadados das páginas *web* são descritos através de meta tags que são interpretadas por computadores, mas que não são apresentadas visualmente aos utilizadores como sendo parte da página. As *meta tags* são particularmen-

te úteis em páginas que contenham pouco texto e deverão ser:

- colocadas na secção <head> do código da página;
- descritivas e adequadas ao conteúdo de cada página;
- mantidas atuais, mesmo em sítios web que sejam atualizados frequentemente.

Os **metadados de uma página em HTML5** mantêm o formato das versões anteriores. Sua sintaxe segue os padrões já abordados e suas opções são:

- **alternate** - um formato alternativo para o conteúdo atual. Por exemplo, para indicar o RSS da página atual usamos:

```
<link rel="alternate" type="application/rss+xml" href="rss.xml" />
```

- **stylesheet** - a folha de estilo "linkada" deve ser vinculada a este documento para exibição
- **Icon**
- **pingback**

Os *links* descritos na página devem conter a descrição dos comportamentos que irão executar. Temos para estes comportamentos as seguintes definições:

- external
- nofollow
- noreferrer

- sidebar
  (W3C)



Link

Aprofunde seus conhecimentos sobre Metadados da página em HTML5. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/HTML/HTML5/HTML5_element_list>. Acesso em: 10 out. 2016.

4. Semântica

Os novos elementos semânticos do HTML5 basearam-se na tentativa de fornecer aos elementos da linguagem uma leitura clara da finalidade daquele elemento dentro do código HTML, ou aproveitando a definição da palavra semântica, procuraram deixar claro o significado de determinado elemento, para que ele se destina, através de seu próprio nome. Elementos semânticos descrevem seu significado ou propósito claramente ao navegador e ao desenvolvedor. Por exemplo, as definições de "*header*" e "*footer*" dizem respeito ao cabeçalho e ao rodapé da página, respectivamente.

O conceito de semântica aplicado ao código tem evoluído a cada versão e ganhou força com a chegada do HTML5. Neles, existem alguns novos elementos semanticamente ricos que podem transmitir a finalidade do elemento tanto a desenvolvedores como a navegadores.

O W3C investigou bilhões de páginas da *Web* existentes para descobrir as IDs e nomes de classe que os desenvolvedores já estavam usando. Uma vez que eliminaram div1, div2 etc., chegaram a uma lista de elementos descritivos que já estavam sendo usados e os transformaram nos padrões (W3C, s.d.).

Citam-se alguns dos novos elementos semânticos em HTML5:

- article
- aside
- figcaption
- figure
- footer
- header
- hgroup
- mark
- nav
- section
- time

Quando se usa **semântica** no código HTML5, por exemplo, usamos uma *tag header* que carrega em si o significado de representar um cabeçalho (CAELUM, s.d).

Link

Você pode encontrar bons exemplos de sobre o que é semântica em HTML em: <http://www.w3c.br/Padroes/WebSemantica>. Acesso em: 10 set. 2016.

Para saber mais

A palavra semântica tem a ver com o significado de determinada palavra ou frase, e no caso do HTML5 este conceito foi empregado para dividir os assuntos ou funções de terminados elementos por nomes que pudessem identificar facilmente qual é a função daquele elemento ou trecho de código dentro de uma codificação de página em HTML5, ajudando seu desenvolvimento e até facilitando o seu entendimento pelos desenvolvedores.

Em todas as páginas da *Web* existem divisões básicas referentes aos tipos de conteúdo que são colocados em cada parte do *layout*, como cabeçalho, rodapé ou menu de navegação.

Nas versões anteriores do HTML não havia *tags* com uma semântica apropriada para cada uma dessas divisões. Dessa forma, os desenvolvedores acabavam usando a tag para todas as situações, e criando seus próprios padrões de nomenclaturas através dos atributos **id** ou **class**.

No HTML5 foram criadas diversas *tags* semânticas para indicar aos *user-agents* quais conteúdos estão sendo inseridos em cada uma das divisões da página, organizando e padronizando o desenvolvimento.

Glossário

Meta tags: *meta tags* são linhas de código HTML ou "etiquetas" que, entre outras coisas, descrevem o conteúdo do seu site para os buscadores da internet (INFOWESTER, 2016).

Ranking: Posicionamento ou colocação dentro dos motores de busca da *web*.

Questão para reflexão

Você conheceu neste módulo as principais *tags* de metadados e também sobre semântica aplicada ao código HTML5. Sendo assim, faça uma pesquisa sobre os principais metadados utilizados em uma página HTML5 a como os motores de busca se utilizam destes metadados para compor o ***Ranking*** de pesquisa, ou seja, como as páginas são classificadas de acordo com estes metadados e como elas podem ser apresentadas em primeiro lugar ou não, dentro de uma pesquisa feita pelo usuário, usando o Google, por exemplo.

Este é um conceito amplamente visto por desenvolvedores e bastante procurado por todas as empresas que pretendem realizar negócios pela internet. Aprofunde seus conhecimentos neste ponto e seu retorno será imensurável.



Considerações Finais

Neste módulo, você aprendeu:

- O que são metadados em HTML5
- Metadados de navegação
- Metadados da página
- Semântica das páginas

Referências

CAELUM. Semântica HTML5. Disponível em: <http:// https://www.caelum.com.br/apostila-html-css-javascript/html-semantico-e-posicionamento-no-css/#3-4-html-semantico>. Acesso em: 10 out. 2016.

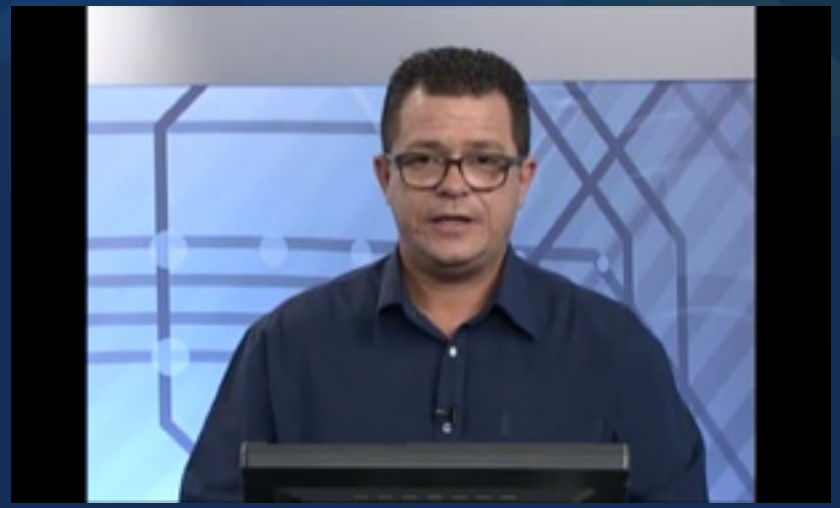
MOZILA. Disponível em: <https://developer.mozilla.org/pt-BR/>. Acesso em: 10 out. 2016.

TABLELESS. Estrutura e Semântica HTML5. Disponível em: <http://tableless.com.br/html5-estrutura-semantica/>. Acesso em: 10 out. 2016.

UXDESIGN. Metadados e estrutura semântica HTML5. Disponível em: <http://www.uxdesign.blog.br/front-end/html5-estrutura-semantica/>. Acesso em: 10 out. 2016.

W3. Site do W3 Org. Disponível em: <https://www.w3.org/>. Acesso em: 10 out. 2016.

Assista a suas aulas



Aula 8 - Tema: Trabalhando com metadata e semântica. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/8f845b35a17efe403f8b62cefc4f5cf0>.



Aula 8 - Tema: Trabalhando com metadata e semântica. Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/2202de9b3f9fe097910c9b7df1b927d1>.

Questão 1

1. Assinale a alternativa que contempla os dois tipos de elementos que existiam inicialmente para gerar um *link* no HTML5.

a) A tag <a> e o elemento base.

b) A tag <a> e a tag <meta>.

c) A tag <a> e a tag <noscript>.

d) A tag <a> e o elemento script.

e) A tag <a> e o elemento link como <link rel=”stylesheet” href=”estilo.css”>.

Questão 2

2. No HTML5, o elemento de navegação é inserido em HTML5 através da *tag* <nav>, que proporciona:

a) *Links* internos no *site* para as demais páginas.

b) *Links* externos no site para as páginas que serão inseridas no *site*.

c) A barra de navegação do *site* e que será repetida nas demais páginas.

d) Um histórico de navegação do *site*.

e) Um mapa de navegação do *site*.

Questão 3

3. Assinale a alternativa que apresenta a função do metadado stylesheet no HTML5.

a) Definir o estilo da barra de navegação do *site*.

b) Informar aos mecanismos de busca a qual categoria pertence o *site*.

c) Definir o estilo de cores do *site*.

d) Chamar a folha de estilo “linkada” que deve ser vinculada ao documento para a exibição.

e) Definir o estilo de fundo (*background*) da página.

Questão 4

4. Assinale a alternativa que contempla os elementos semânticos definidos pelo HTML5:

a) nofollow, figcaption, header, footer.

b) aside, figcaption, header, footer.

c) aside, external, header, sidebar.

d) sidebar, , header , figcaption, external.

e) external, nofollow, header, footer.

Questão 5

5. Em HTML5, a semântica foi pensada e empregada no sentido de:

a) Transmitir a finalidade do elemento tanto a desenvolvedores como a navegadores.

b) Identificar a semântica de conteúdo das páginas.

c) Os mecanismos de busca classificarem os sites de acordo com a semântica.

d) Desenvolver novos significados para elementos do HTML.

e) Aplicar semânticas diferenciadas em cada página de um *site*.

Gabarito

1. Resposta: E.

Para gerar um *link* entre páginas inicialmente existiam dois tipos de elementos, a tag *<a>* ou o elemento *link*, demonstrados nos códigos a seguir:

```
<a href="http://www.agenciatur.
com.br">Visite a página de viagens
</a>
<link rel="stylesheet"
href="estilo.css" />
```

2. Resposta: C.

O elemento de navegação é inserido em HTML5 através da *tag* <nav>, que marca a seção na página que aponta para outras páginas, basicamente a seção onde é descrita a barra de navegação do *site* e que será repetida nas demais páginas.

3. Resposta: D.

A folha de estilo "linkada" deve ser vinculada ao documento para exibição através do metadado de página *stylesheet*.

4. Resposta: B.

As demais opções contêm pelo menos um elemento que não pertence às definições de semântica da linguagem.

5. Resposta: A.

O conceito de semântica aplicado ao código tem evoluído a cada versão e ganhou

força com a chegada do HTML5. Neles, há alguns novos elementos semanticamente ricos que podem transmitir a finalidade do elemento tanto a desenvolvedores como a navegadores.

